SUP'COM

المـــدرســـة العليـــا للمـــواصـــلات بتونـــس
ÉCOLE SUPÉRIEURE DES COMMUNICATIONS DE TUNIS

Cycle de Formation des Ingénieurs en Télécommunications

Option : Services Numériques et Intelligence Artificielle

Rapport de Projet de Fin d'Études

Création d'un Agent Conversationnel Intelligent

Réalisé par :

Siwar Bel Haj Mohamed

Encadrants :

Mme. Ikram Chaabane (3S)

Mme. Ines Moussa (Sup'Com)

Organisme d'accueil :



Année universitaire : 2020/2021

Signature de rapport

Encadrant Professionnel :
Mme Ikram Chaabane
Chef de projet à Standard Sharing Software.

Encadrant Académique :
Mme Ines Moussa
Professeur à Sup'COM.

Dédicaces

À mes chers parents,

Ma raison d'être,
Les mots me manquent pour exprimer toute la gratitude et l'amour profond que j'ai pour les sacrifices que vous avez faits pour ma réussite et pour vos encouragements tout au long de mon parcours
pour que je sois à cette étape de ma vie.
J'espère que vous trouviez ici le témoignage de ma gratitude, de ma reconnaissance et de mon respect, que dieu vous bénisse avec une bonne santé et une longue vie.

À mes chères soeurs, Sirine & Sinda,

A mes flammes d'espoir.
Pour qui je sais que mon succès est très important, que Dieu vous protège et vous donne une vie pleine de bonheur et de succès.

À tous ceux qui me sont chers...,

Remerciements

L'accomplissement de ce stage est le résultat de la contribution et de l'encadrement efficace d'innombrables personnes qui m'ont aidés au cours de ce projet.

Tout d'abord, je tiens à remercier Mme Ikrame Chaabane pour son soutien continu, sa supervision quotidienne et ses mots sages durant ce stage. Le succès de ce stage est dû à sa volonté de me pousser à faire de mon mieux.

Mes remerciements vont droit au cœur à Mme Iness Moussa, ma tutrice académique, pour sa disponibilité et ses mots d'encouragement.

J'exprime, en plus, ma vive reconnaissance à tous les enseignants de l'École Supérieure des communications de Tunis ''Sup'Com", qui m'ont soutenu tout au long de ce chemin.

Mes vifs remerciements vont de même aux membres du jury pour avoir pris le temps d'évaluer mon travail et de m'avoir donné de précieux commentaires.

Enfin, un grand merci à toutes les personnes qui ont contribué directement ou indirectement au projet et qui n'ont pas été explicitement mentionnées. Vos aides et vos soutiens sont très appréciés.

Résumé

Avec la croissance rapide des techniques de traitement automatique du langage naturel (TALN), plusieurs entreprises cherchent des solutions de chatbot automatisées pour fournir des services de qualité à une base d'utilisateurs croissante.

L'objectif de ce travail est de contribuer à l'implémentation de la solution par la mise en place d'un chatbot consacré à automatiser toutes les fonctions de l'assistant chez « Globalnet » telles que la vérification des factures et des contrats, le suivi de l'état des demandes de client et la consultation des moyens de paiement. On a conçu et réalisé un modèle de classification d'intention qui combine les méthodes de traitement automatique des langues, l'efficacité de l'apprentissage par transfert et la capacité de distinguer les différents besoins des clients.
Finalement, on a intégré un modèle d'analyse sentimentale afin de mesurer la satisfaction du client. Ce projet a donc pour but de montrer l'importance des technologies et de l'intelligence artificielle (IA) dans le service client.

Mots clés : TALN, Apprentissage par transfert, Camembert, Classification d'intention, Analyse sentimentale, Pytorch, Interface conversationnelle, Service client.

Abstract

With the rapid growth of Natural Language Processing (NLP) techniques, several companies are looking for automated chatbot solutions to provide quality services to a growing user base.

The aim of this work is to contribute in the implementation of an automated solution by setting up a chatbot dedicated to automating all the functions of the assistant at Globalnet as checking invoices and contracts, monitoring the status of customer requests and consulting payment methods. We designed and implemented an intent classification model that combines automatic language processing methods, the efficiency of transfer learning and the ability to distinguish between different customer needs.
Finally, We integrated a sentimental analysis model to measure customer satisfaction.
This project highlights the capabilities of technologies and the impact of artificial intelligence (AI) on customer service.

Key words : NLP, Transfer learning, CamemBERT, Intents classification,Sentimental analysis, Pytorch, Conversational interfaces ,Customer service.

"Focusing On the customer makes a company more resilient.".Jeff Bezos, CEO @ Amazon

Table des Matières

Table des Matières **x**

Liste des Figures **xiii**

Liste des Tableaux **xiv**

Liste des Abréviations **xv**

Introduction générale **1**

1 Cadre Général du Projet **3**

1.1 Introduction 3

1.2 Présentation de l'organisme d'accueil 3

1.2.1 Les cinq principaux atouts qui font la force de **3S** 3

1.2.2 Parcours de 3S 4

1.3 Présentation du projet 4

1.3.1 Problématique 5

1.3.2 Étude de l'existant 5

1.3.3 Solution proposée 6

1.4 Méthodologie de travail 6

1.4.1 Les phases de la méthodologie CRISP 7

1.5 Déroulement du projet 8

1.6 Conclusion 9

2 État de l'Art **10**

2.1 Introduction 10

2.2 Aperçu du système de Chatbot . . . 10
2.2.1 Type de chatbots en interaction client . . . 10
2.2.2 Méthodologies d'implémentation d'un chatbot . . . 12
2.3 Le traitement du langage naturel(NLP) . . . 13
2.3.1 Importance du NLP . . . 13
2.3.2 NLU (Compréhension du langage naturel) . . . 13
2.3.3 La classification d'intention . . . 14
2.4 les bases de NLP . . . 14
2.4.1 Tokenisation . . . 14
2.4.2 La morphologie . . . 15
2.4.3 Vectorisation du texte . . . 15
2.4.3.1 Bag of words (BoW) . . . 16
2.4.3.2 TF-IDF « Term Frequency-inverse document frequency » 16
2.4.3.3 Word2Vec . . . 17
2.5 L'apprentissage automatique . . . 18
2.5.1 SVM . . . 18
2.5.2 Naive Bayes . . . 19
2.5.3 Random forest . . . 19
2.5.4 Logistic regression . . . 20
2.6 L'apprentissage profond . . . 21
2.6.1 Layer . . . 21
2.6.2 Forward propagation et back propagation . . . 21
2.6.3 Fonction d'activation . . . 21
2.6.4 Fonctions de perte . . . 22
2.6.5 Multi-layer perceptrons . . . 23
2.6.6 RNN . . . 23
2.6.7 LSTM . . . 24
2.6.8 Bidirectional-LSTM . . . 25
2.7 Apprentissage par transfert . . . 25
2.8 Transformers . . . 26
2.8.1 BERT . . . 27
2.8.2 CamemBERT . . . 28

2.9 Conclusion 29

3 Prétraitement des Données et Modélisation 30

3.1 Introduction 30
3.2 Classification d'intention de l'utilisateur 30
3.2.1 Description de l'ensemble de données 30
3.2.2 Étiquetage de données 31
3.2.2.1 Similarité cosinus 31
3.2.3 Préparation et pré-traitement des données 32
3.2.4 Augmentation des données 35
3.2.4.1 Pré-traitement BERT 37
3.2.4.2 Prétraitement CamemBERT 38
3.2.5 Analyse exploratoire des données du modèle de classification d'intention 39
3.2.6 Formulation du problème : Classification de textes multi-classes . 42
3.2.7 Modélisation 43
3.2.7.1 Application de divers modèles de machine learning . . . 43
3.2.7.2 Application de divers modèles de deep learning 43
3.2.8 Application de divers modèles de transfert learning (BERT et CamemBERT) 45
3.3 Analyse sentimentale 46
3.3.1 Importance de l'analyse sentimentale pour un chatbot 46
3.3.2 Description des Données 47
3.3.3 Analyse exploratoire des données 48
3.3.4 Modélisation 49
3.4 Environnement de travail 49
3.5 Conclusion 51

4 Résultats Expérimentaux des Modèles et Discussions 52

4.1 Introduction 52
4.2 Métriques d'évaluation 52
4.3 Résultats et discussions 54
4.3.1 Classification contextuelle et classification d'intention 54
4.3.1.1 Approche machine learning 54

4.3.1.2 Approche deep Learning 55
4.3.1.3 Approche transfert learning 57
4.3.2 Analyse sentimentale 59
4.3.2.1 Résultats et discussions : analyse sentimentale 60
4.4 Visualisation 61
4.4.1 Classification contextuelle et classification d'intention 61
4.4.2 Analyse sentimentale 66
4.5 Conclusion 67

5 Développement de L'interface Conversationnelle 68
5.1 Introduction 68
5.2 Analyse et spécification des besoins 68
5.2.1 Identification des acteurs 68
5.2.2 Spécifications des besoins 69
5.2.3 Conception et modélisation UML 70
5.2.3.1 Diagramme de cas d'utilisation 70
5.2.3.2 Diagramme de communication 71
5.2.3.3 Diagramme de séquence 71
5.2.3.4 Diagramme d'activité : Demande un nouveau mot de passe 73
5.3 Déploiement 74
5.3.1 Le Système de gestion de base de donnée 75
5.3.2 Environnement de développement 75
5.3.3 Liaison entre la partie Backend et Frontend 76
5.3.4 La liaison entre le chatbot et la base des données de Globalnet . . 77
5.4 Architecture de la solution finale 78
5.5 Conclusion 79

Conclusion Générale 80

Annexe 82

Bibliographie 88

Liste des Figures

1.1 Logo de l'entreprise 3s 4
1.2 Cycle de vie de la méthodologie CRISP 8
1.3 Diagramme de Gantt 9

2.1 NLU et NLP 14
2.2 Stemming et lemmatisation 15
2.3 Architecture de l'approche CBOW et Skip-Gram 17
2.4 Séparation des données à l'aide d'un hyperplan 18
2.5 Architecture Random Forest 20
2.6 Architecture Multi-layer perceptrons 23
2.7 Architecture réseau de neurones récurrents. 24
2.8 Illustration de LSTM. 24
2.9 Illustration de BILSTM. 25
2.10 Illustration du processus d'apprentissage par transfert 26
2.11 Architecture du transformers 26
2.12 Architecture BERT 28

3.1 Pipeline de nettoyage de données. 32
3.2 Un e-mail avant pré-traitement 33
3.3 Les données avant la phase de prétraitement 34
3.4 La base d'apprentissage après la phase de prétraitement 34
3.5 Données collectées 35
3.6 Des classes déséquilibres 37
3.7 Exemple de pré-traitement de texte en utilisant mBERT. 38
3.8 Exemple de pré-traitement de texte. 39

3.9 Distribution des échantillons par intention dans l'ensemble de données de la classe information générale 39
3.10 Distribution des échantillons par intention dans l'ensemble de données de la classe commerciale 40
3.11 Distribution des échantillons par intention dans l'ensemble de données de la classe technique 40
3.12 Distribution des échantillons par intention dans l'ensemble de données de la classe financière 40
3.13 Nombre de token par question 41
3.14 Classe demande de payement 41
3.15 Classe pas de connexion 42
3.16 Classifications LSTM à 4 contextes. 44
3.17 Architecture du modèle Bidirectional-LSTM 45
3.18 Étapes de conception et de mise en œuvre d'un modèle d'analyse sentimentale. 47
3.19 Des commentaires d'une enquête de satisfaction 47
3.20 Répartitions des données. 48
3.21 Nombre de tokens par chaque classe 48

4.1 Matrice de confusion Pour classification Binaire. 53
4.2 Architecture de modèle d'analyse sentimentale CamemBERT. 61
4.3 Évolution de la perte de formation et de validation 61
4.4 Relation entre (F1-score/accuracy) et époques. 62
4.5 Accuracy pour différentes tailles du lot 63
4.6 Évaluation du modèle CamemBERT : Matrice de confusion 63
4.7 Matrice de confusion normalisée montrant toutes les entités de l'ensemble de données de classe commerciale pour le modèle CamemBERT 64
4.8 Classe Commerciale utilisant mBert 65
4.9 Visualisations des données de formation à travers les couches d'un modèle CamemBERT 66
4.10 CamemBERT AUC ROC 66
4.11 Naïve Bayes AUC ROC 67

5.1 Diagramme de cas d'utilisation global. 70
5.2 Diagramme de communication. 71

5.3 Diagramme de séquence. 72
5.4 Diagramme d'activité : demande un nouveau mot de passe 74
5.5 Architecture logiciel du projet 77
5.6 Le résultat d'accès au base des données de Globalnet 78
5.7 Système agent intelligent 78
5.8 L'agent Virtuel G-nett 79
A.1 CamemBERTForSequenceClassification 82
A.2 BERTForSquenceClassification 83
A.3 Pour chaque mot Xi on obtient trois vecteurs associés au vecteur Qi, Ki, Vi 84
A.4 Visualisation mécanisme de self attention 85

Liste des Tableaux

3.1 Architecture des différents modèles de mBERT 45

4.1 Comparaison des performances des classifieurs statistiques 55
4.2 Résultats de modèle traditionnel de deep Learning "LSTM" 56
4.3 Résultats de modèle traditionnel de deep Learning "BILSTM" 56
4.4 Meilleurs hyperparamètres de modèle mBERT 57
4.5 Résultats du modèle de langage mBERT 57
4.6 Meilleurs hyperparamètres de modèle CamemBERT 58
4.7 Résultats du modèle de langage CamemBERT 58
4.8 Rappel, Précision et F1-score du modèle CamemBERT 59
4.9 Résultats des tests du modèle d'analyse sentimentale. 60

Liste des Abréviations

IA	Artificial intelligence
BERT	Bidirectional Encoder Representations from Transformers
ROBERTA	Robustly Optimized BERT
CBOW	Continuous Bag-OF-Words
NLP	Natural Language Processing
NLU	Natural Language Understanding
TFIDF	Term Frequency-Inverse Document Frequency
ReLU	Rectified Linear Unit
RNN	Recurrent Neural Network
DSL	Domain-specific language

RL	Logistic regression
RF	Random Forest
SVM	Support Vector Machine
DL	Deep Learning
AUC	Area Under the Curve
ROC	Receiver Operating Characteristic
API	Application Programming Interface

Introduction générale

L'amélioration du service client est au cœur d'une entreprise qui vise à renforcer ses relations avec ses clients tout en assurant une assistance de qualité aux inquiétudes annoncées.

Ainsi, pour fournir un service client de qualité et maintenir sa position concurrentielle, les entreprises auront besoin d'augmenter le nombre d'employés ce qui nécessite des ressources matérielles et financières supplémentaires. Pour cette raison, l'intelligence artificielle est intégrée dans les stratégies des entreprises afin de mieux répondre aux besoins de ses clients à travers l'utilisation des messages automatisés.

La concrétisation de ces objectifs passe par l'intégration des agents virtuels intelligents appelés chatbots.

Un chatbot est un logiciel informatique, généralement piloté par l'intelligence artificielle, qui permet à un client de communiquer avec une entreprise ou une marque dans un format textuel et/ou vocal. En fait, il simule la messagerie instantanée entre humains. Il est souvent intégré dans les sites web ainsi que dans les applications mobiles des entreprises. Plus précisément, le chatbot permet d'effectuer de nombreuses tâches en donnant l'impression qu'il peut comprendre, agir et parler instantanément.

Les chatbots ont suscité beaucoup d'attention et d'intérêt ces dernières années grâce à l'évolution de l'IA, d'apprentissage automatique et aux défis comme « le prix Loebner ».
L'objectif principal de ce prix est de créer des chatbots à l'aspect humain. À présent, les entreprises viennent de commencer à apprendre à les utiliser efficacement dans leurs industries telles que l'éducation, la banque, le commerce électronique, la santé, la nourriture, les jeux et en particulier dans le service client. Les chatbots alimentés par l'IA sont devenus largement disponibles et nous ont permis de réaliser ce qui était autrefois impossible tels que aider les clients 24h / 24 et 7j / 7, résoudre automatiquement les questions sans aucune intervention humaine et fournir une assistance à plusieurs clients à la fois.
Les chatbots sont devenus indispensables des opérations des services clients des entreprises afin d'assurer leurs satisfaction.

Des statistiques récentes montrent que plus de 265 milliards de demandes d'assistance client sont effectuées chaque année et qui coûtent très chères aux entreprises (plus que 1.3 billion de dollars) pour les traiter. Les chatbots peuvent réduire ces coûts de manière significative lorsque les entreprises passent d'une technologie IVR désuète et inefficace à l'IA, plus précisément aux chatbots qui contribuent déjà à transformer les centres d'appels à travers le monde. [1]

Pour répondre à cette problématique, notre stage a pour objectif d'exploiter et d'appliquer les nouvelles techniques de l'intelligence artificielle dans la communication. Plus précisément, notre mission consiste à assurer la satisfaction des clients d'une part, et d'autre part d'obtenir une bonne expérience client à travers le développement d'un agent conversationnel à intelligence artificielle. Cet assistant virtuel a pour rôle d'utiliser les nouvelles techniques de l'IA afin de fournir une assistance à haute valeur ajoutée aux services client.

Pour cette raison, ce travail sera divisé en 5 chapitres :

• Le premier chapitre est consacré à la présentation du cadre général du projet.

• Dans le deuxième chapitre, on va présenter dans une première partie les chatbots puis les différents concepts liés à notre projet.

• Dans le troisième chapitre, on va présenter les démarches d'élaboration de notre chatbot intelligent, les données utilisées pour la modélisation. On va présenter par la suite les choix technologiques.

• Dans le quatrième chapitre, on va discuter les résultats des modèles obtenus.

• Dans le cinquième chapitre, on va présenter le développement d'une interface conversationnelle et on va illustrer sa mise en œuvre.

Et on achève notre projet par une conclusion générale décrivant le travail réalisé et les perspectives envisagées.

Chapitre 1

Cadre Général du Projet

1.1 Introduction

Ce premier chapitre du rapport donne une vision globale du projet. On présentera dans une première partie l'entreprise d'accueil **«Standard Sharing Software (3 S)».** Ensuite, on élaborera la problématique qui nous amène à implémenter le chatbot « G-nett», puis on va présenter la solution. Enfin, on va détailler la méthodologie utilisée.

1.2 Présentation de l'organisme d'accueil

3S est une entreprise phare du secteur de la technologie, entreprise agile avec une histoire de plus de 30 ans en Tunisie et en Afrique du nord. Elle s'est positionnée avec succès comme un leader sur le marché des services d'intégration. **3S** est pionnier dans un marché en évolution rapide, elle possède plus que 370 compétence de haute qualité et expérimenté, dont près des deux tiers sont des ingénieurs ou des techniciens avec plus de 300 certifications.

1.2.1 Les cinq principaux atouts qui font la force de 3S

Les principaux atouts du **3S** sont les suivants :

1. Maîtrise des technologies émergentes.
2. Expertise technique certifiée par les normes internationales.
3. Couverture technique étendue pour fournir des solutions de bout en bout.
4. Support technique basé sur des procédures de résolution des incidents efficaces.
5. Des bases financières solides.

1.2.2 Parcours de 3S

Le parcours de **3S** est marqué par une série de certifications prestigieuses, telles que : VMware, Microsoft, Cisco, Fortinet, EMC, IBM, etc.
C'est également la seule entreprise en Tunisie à avoir obtenu la certification «Cisco Silver Partner».
3S a créé sa filiale GlobalNet en 1997, qui est devenue un fournisseur d'accès internet. En 2007, l'entreprise est devenue le premier partenaire tunisien de "Cisco Systems". Ce partenariat s'est imposé avec succès comme un acteur majeur dans le domaine des réseaux informatiques. Outre que GlobalNet, **3S** possède d'autres filiales œuvrant dans le domaine d'informatique

On peut citer :

- L'instar de Technical Training.
- Get Wireless.
- NX Tunisie.
- SmartUP.
- High Tech.
- Pro2C.
- Cyberesa [4].

La figure 1.1 présente le logo de l'entreprise.

FIGURE 1.1 – Logo de l'entreprise 3s
[10]

1.3 Présentation du projet

Après avoir présenté l'entreprise **3S**, on consacrera cette section à présenter le projet.

1.3.1 Problématique

La majorité des consommateurs considèrent un bon service client comme une raison de s'en tenir à une entreprise. En effet, un produit ou un service unique peut les attirer, mais le service client les garde.

Par ailleurs, la quantité de communication effectuée à travers les interactions textuelles, dépasse infiniment la communication vocale. C'est le cas également de la communication entre les entreprises et les clients : cela montre l'immense potentiel des agents virtuels.

Dans ce sens, la mission principale du service clientèle de Globalnet est la gestion des flux de réclamations répétitives, avec une insuffisance personnelle qui peut répondre aux besoins du client tout en respectant le délai. En outre, l'entreprise souhaite prendre en charge plus de canaux de communication (web, Facebook messenger, Whats App, Skype...etc.) sans besoin d'agrandir l'équipe. De surcroît, il y a des périodes où les assistants peuvent avoir des difficultés pour gérer une quantité énorme des messages. De même, ça influence le mécontentement de client. Ainsi, le besoin d'être en contact avec le client et répondre à ses demandes en dehors des heures de travail.
Afin de mesurer le degré d'efficacité de ses services l'agent doit lancer à chaque fois une enquête de satisfaction, mais le résultat n'est pas toujours lisible et pertinent.

Devant ce contexte dilemmatique, plusieurs facteurs qui sont réunis afin d'assurer une migration du cadre de service client traditionnel chez Globalnet a une maturité numérique, c'est-à-dire de mettre en œuvre un chatbot intelligent. Mais cette tendance met en évidence beaucoup de questions et de besoins auxquels il faut répondre :

1. Comment construire un système qui facilite l'accès aux informations et aux données personnelles et qui assure la gestion de ses services ?
2. Comment garantir la sécurité et la confidentialité d'un espace clientèle virtuel ?
3. Comment assurer un partage et un flux de données fiable et bien contrôlé ?
4. À quel niveau et à quelle portée pouvons-nous réaliser l'automatisation des processus des services clientèles de Globalnet quelque soit technique, financier ou commercial et quelle est la meilleure façon d'utiliser et d'exploiter l'intelligence artificielle et plus précisément le NLP pour assurer l'optimisation et la robustesse de l'automatisation du système et du service client ?

Donc de nos jours, il y a une nécessité fondamentale de fournir des services clientèle digitalisés et automatisés afin d'assurer un service de haute performance.

1.3.2 Étude de l'existant

L'agent virtuel intelligent est parmi les innovations majeures de ces dernières années.
Pour cela, il était rapidement intégré dans la stratégie des entreprises à travers le monde.

Les attentes des clients augmentent chaque jour. D'où l' apparition de nouvelles technologies qui permettent de rendre le service clientèle plus à jour, plus rapide, et plus fiable. Tels que le cas de Tunisie, l'intégration de ce agent dans le service clientèle devient une étape nécessaire, en effet l'utilisation de ces derniers a augmenté énormément dans l'industrie dans les dernières années. Citons les exemples suivants : l'agent virtuel d'Amen Bank « AmenBot », le Chatbot d'Ooredoo « Wajih » et d'Orange « Djingo el Damdoum ». Dans le contexte de notre projet, nous nous sommes intéressés par le chatbot de Topnet puisque cet agent virtuel intervient dans le même secteur que notre solution.

Le chatbot de TOPNET "Aminos" :

Le chatBot Aminos est un agent virtuel basé sur un menu. Le but d'un tel chatbot est de guider un utilisateur à travers une arborescence de scénarios prédéfinie. Autrement-dit, ce type de chatbot ressemble à un système IVR basé sur du texte. Tout comme un IVR, un chabot basé sur un menu offre des choix et demande des données. Habituellement, il dit : "Je suis un agent qui peut vous aider avec les problèmes suivants, veuillez appuyer sur le bouton correspondant : X, Y».

Les scénarios d'un tel chatbot sont simples et rigides basés sur des boutons ou sur la récupération du mot clé. Ils ne peuvent pas automatiser toutes ces tâches complexes. Il faut vraiment la flexibilité du langage naturel afin de capturer une grande variété d'intention des utilisateurs, collecter les données nécessaires et fournir une assistance.

1.3.3 Solution proposée

Notre solution consiste à implémenter un agent intelligent qui a la capacité d'automatiser une communication simultanée et complexe avec plusieurs clients afin de réduire la charge sur l'entreprise.
Un chatbot d'assistance technique, commerciale et financière, qui peut vérifier les factures et les contrats du client, il peut répondre aux demandes les plus fréquentes, consulter les différentes promotions du Globalnet, sauvegarder les différentes conversations et assigner à la fin une remarque à propos l'état du client satisfait, insatisfait.
La fin de la conversation avec notre chatbot peut être suivie d'une conversation téléphonique avec un agent de support et même de finaliser par un e-mail de confirmation afin de préciser l'état d'une demande. Au lieu d'être une solution autonome, notre agent virtuel devient une partie intégrante d'un écosystème communicationnel diversifié.

1.4 Méthodologie de travail

Une méthode de travail est une démarche à suivre pour réussir un projet. Elle est utilisée pour structurer, assurer la conception et contrôler la réalisation d'un projet.

La méthodologie adaptée au cours de la réalisation de ce système est basée sur la méthodologie CRISP (Cross Industry Standard Process).
Cette méthodologie est la clé de réussite en data science. Elle fournit une approche structurée de planification de projet d'exploration de données. De plus, elle est très robuste et éprouvée. La méthodologie CRISP comporte six étapes allant de la compréhension du problème métier au déploiement et la mise en production.

1.4.1 Les phases de la méthodologie CRISP

1. La compréhension du problème métier :

   La première étape consiste à bien comprendre les éléments métiers et problématiques que la data science vise à résoudre ou à améliorer. C'est une étape très importante pour notre projet.

2. La compréhension des données :

   L'objectif de cette phase est de savoir ce qui peut être attendu et réalisé à partir des données. Elle commence tout d'abord par une collecte de données nécessaires, puis par une identification de la qualité des données disponibles. Cette phase est liée à la phase précédente.

3. Préparation et pré-traitement des données :

   Cette étape comprend toutes les activités de préparation de l'ensemble des données finales à partir des données initiales. Au cours de cette phase on a intérêt à transformer les données en données reconnues par les algorithmes et les processus. En effet, La qualité des données nettoyées aura un impact sur les performances du modèle. Pour cette raison, Les tâches de préparation des données sont susceptibles d'être effectuées plusieurs fois et dans un ordre non prescrit raison pour laquelle on a consacré beaucoup de temps pour cette tâche.

4. La modélisation :

   La modélisation est au cœur de tout projet d'apprentissage automatique, à ce stade, diverses techniques de modélisation sont sélectionnées et appliquées. Cette phase comprend le choix, le paramétrage et le test de différents algorithmes.
5. L'évaluation :

   Cette phase vise à vérifier le modèle ou les connaissances obtenues afin de s'assurer qu'ils répondent aux objectifs formulés au début du processus. Dans le cas où les résultats sont erronés, la méthodologie permet de revenir à la première étape, afin de comprendre pourquoi les résultats sont erronés. À ce stade, on teste notamment la robustesse et la précision des modèles obtenus.
6. Le déploiement

   L'étape finale du processus, consiste à présenter les résultats d'une manière utile et compréhensible pour les utilisateurs finaux des modèles obtenus.



FIGURE 1.2 – Cycle de vie de la méthodologie CRISP [23]

Dans la section suivante, on va présenter les différentes étapes de déroulement du projet après avoir détaillé la méthodologie adaptée au cours de ce travail.

1.5 Déroulement du projet

Le diagramme de Gantt est un outil de planification qui permet de gérer un projet, de suivre la progression d'un plan complet. Il représente la liste des tâches qui doivent être

effectuées pendant la durée de vie d'un projet. Les activités sont présentées sous forme de barres. La longueur de chaque barre donne une idée sur la date de début et de fin, ainsi que sur la durée de cette tâche.
Le diagramme 1.3 représente les principales tâches à réaliser au cours du déroulement du projet.

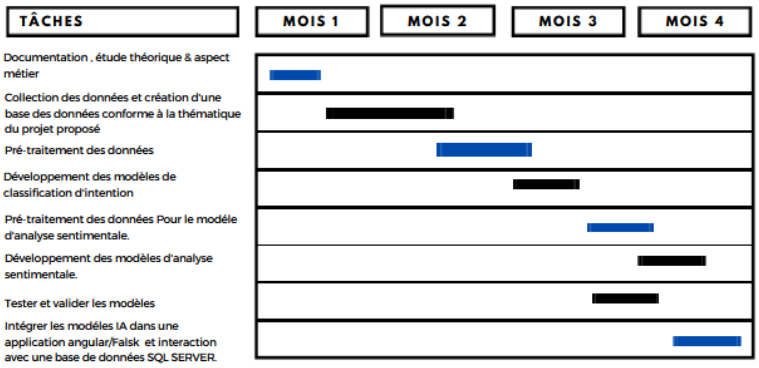


FIGURE 1.3 – Diagramme de Gantt

Grâce à ce diagramme on a pu organiser toutes les étapes de la réalisation de notre solution, de comprendre les dépendances des tâches et de terminer le projet dans les délais impartis.

1.6 Conclusion

A travers ce premier chapitre, on a présenté dans une première partie le cadre de projet de fin d'études en présentant l'entreprise d'accueil, puis on a présenté la problématique accompagnée d'une étude de l'existant. On a clôturé ce chapitre par une description de la solution proposée tout en mettant en évidence le travail qui a été effectué durant ce stage.
Les différentes approches théoriques liées à notre sujet seront abordées dans le chapitre suivant.

Chapitre 2

État de l'Art

2.1 Introduction

Dans ce chapitre, on va présenter une étude préliminaire du projet afin d'en assurer une compréhension approfondie. Tout d'abord, on va présenter les concepts de base du traitement de langage naturel, puis on va exposer brièvement les différents types de modèles afin de choisir le modèle le plus adéquat aux spécifications du projet.

2.2 Aperçu du système de Chatbot

Pour développer un agent virtuel, il existe plusieurs outils de développement. Ces derniers peuvent être divisés en deux catégories principales :

2.2.1 Type de chatbots en interaction client

1. Chatbots de menu :

   Ce sont les types de Chatbots les plus utilisés dans le marché, basé sur des hiérarchies des arbres de décisions présentées aux utilisateurs sous la forme de menu. Pour donner la réponse finale, les utilisateurs doivent effectuer plusieurs sélections guidées par ce chatbot.

2. Chatbots basés sur des règles :

   Ils fonctionnent comme des FAQ interactives. Ils sont programmés pour reconnaître certains termes, à partir desquels ils peuvent répondre à l'utilisateur final en utilisant des réponses prédéfinies. Ce type de chatbot suit les mots-clés pour

déterminer comment offrir la meilleure réponse.

3. Chatbots IA :

   Ces chatbots agissent comme des cerveaux artificiels et utilisent des capacités sophistiquées de traitement cognitives et naturelles du langage. Les chatbots IA peuvent comprendre les demandes ainsi que le contexte, l'intention et les émotions, en utilisant des techniques d'apprentissage automatique et d'apprentissage en profondeur pour devenir plus intelligents et mener des conversations plus élaborées en temps réel. Ils sont suffisamment intelligents pour progresser et s'améliorer en fonction de ce que les utilisateurs demandent et de la manière dont ils le demandent.

2.2.2 Méthodologies d'implémentation d'un chatbot

Méthodologie	Avantages	Désavantages	Exemples
Plateforme spécialisé	-Nécessite aucune connaissance en programmation ou de capacité de développement. -Courte durée de développement. -S'intégrer facilement à des applications tierces.	-Ne peuvent pas offrir le niveau de service que les clients attendent. -Pas de capacités d'auto-apprentissage. -Basé sur la reconnaissance des mots clés.	Dialogflow, Chatfuel, Botsify, Motion.ai, Aivo.
Développement Scratch	-Capacité de résoudre des processus très complexes. - Fournir une évolutivité à long terme. -Peut générer une expérience client de haut niveau pour l'entreprise. -Solution est totalement personnalisée. -S'intégrer à d'autres chatbots et fonctionner sur plusieurs plateformes.	-Nécessite des compétences et de connaissance avancé de solutions d'apprentissage automatique et de NLP. -Coûteux et long à construire au départ.	AWS Chatbot

Le NLP est au cœur du fonctionnement des chatbots actuels. C'est pourquoi dans les sections suivantes, on va concentrer sur les principales méthodes utilisées pour le traitement du langage naturel.

2.3 Le traitement du langage naturel(NLP)

Le traitement du langage naturel est la discipline de l'informatique à orientation linguistique qui s'intéresse à la capacité des logiciels à comprendre le langage humain naturel. Grâce à NLP, la communication homme-machine devient de plus en plus facile, car il permet aux machines d'obtenir et de traiter des informations à partir des entrées écrites de l'utilisateur.
Contrairement aux opérations de traitement de texte courant, le NLP ne traite pas la parole ou le texte comme une séquence de symboles. Il prend en considération la structure hiérarchique de la langue naturelle, les mots créent des phrases, les phrases se transforment en idées cohérentes. En d'autres termes, Le NLP ne recherche pas seulement les mots-clés. Il utilise des connaissances préprogrammées ou acquises pour décoder le sens et l'intention à partir de facteurs tels que la structure de la phrase, le contexte et les expressions idiomatiques.

2.3.1 Importance du NLP

Le langage naturel comporte ses propres défis. Non seulement le contenu n'est pas structuré, mais le langage lui-même est ambigu et incohérent. Les métaphores, la polysémie, la rhétorique telle que le sarcasme ou l'ironie et une vaste collection d'ambiguïtés sont même difficiles à saisir pour les humains lors de la lecture. En même temps, les domaines techniques dans lesquels elle est utilisée jouent un rôle essentiel en définissant le sens des mots. Alors, comment le NLP fait-elle face à ces obstacles ?
Traditionnellement, les méthodes employées par les praticiens de le NLP sont basées sur des ensembles des règles complexes écrites à la main. La conception des règles qui tentent de modéliser la complexité d'une langue devrait prendre en compte tous les éléments linguistiques. Ces systèmes sont difficiles à mettre en œuvre, à maintenir, à mettre à l'échelle et à transférer. Ces systèmes représentaient la norme jusqu'à la fin des années 80. Ensuite, la recherche s'est tournée vers l'apprentissage automatique et les méthodes statistiques.
Le NLP est divisée en deux catégories clés : NLU et NLG.
Pour notre projet, on va s'intéresser uniquement par la NLU.

2.3.2 NLU (Compréhension du langage naturel)

La NLU, natural langage understanding en anglais est un sous-ensemble de le NLP, le terme NLU est utilisé pour exprimer la capacité d'une machine à comprendre le langage naturel sous une forme fournie par les humains.
NLU prend du texte en entrée, comprend le contexte et l'intention.
Dans la figure 2.1 met en évidence les différences entre NLU et NLP.

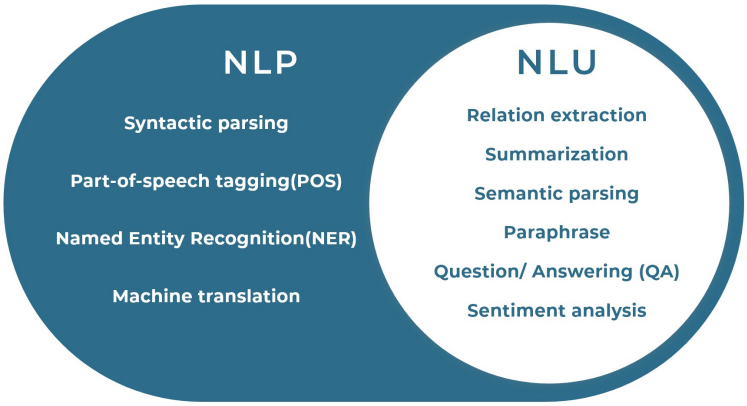


FIGURE 2.1 – NLU et NLP

2.3.3 La classification d'intention

La classification d' intention est un élément important de tout système de compréhension du langage naturel (NLU) et dans toute plateforme de chatbot. Elle consiste à associer les énoncés de l'utilisateur à des significations prédéfinies appelées intentions.
Alors pour que un chatbot fonctionne bien, il doit reconnaître correctement l'intention de l'utilisateur à partir de ses entrées afin de déclencher l'action ou la réponse adéquate.
Exemples, pour la classification d'intention on peut citer :
"Bonjour" "Bonsoir" "Salut" →salutation.
"pourquoi la connexion est toujours suspendue ?" →Connexion Suspendu.
"ma connexion ne fonctionne plus ?"

2.4 les bases de NLP

Dans le domaine de NLP, il existe plusieurs techniques et méthodes qui consistent à prétraiter et transformer les informations en entrée en un jeu de données exploitables parmi ceux on peut citer :

2.4.1 Tokenisation

La tokenisation est la première étape de prétraitement. C'est un moyen de séparer un texte, une phrase en une liste des unités plus petites appelées jetons, les jetons peuvent être des mots, des caractères ou des sous mots. Cela peut se faire avec des règles simples (séparation par des espaces, ponctuation...) mais peut être ainsi sur la base d'un processus statistique (WordPiece Tokenization[33], sentencepiece Tokenization[20]).

2.4.2 La morphologie

La morphologie est le domaine de la linguistique qui analyse la structure interne des mots. Il existe deux approches : «Stemming» et «Lemmatisation».

Stemming

La stemming est l'approche classique en linguistique. Les mots sont constitués de morphèmes qui sont les unités linguistiques non décomposables. Ils sont les porteurs de sens. Avec la radicalisation ces derniers sont réduits à leurs racines de mots. Une racine de mot n'a pas besoin d'être la même racine qu'une racine morphologique basée sur un dictionnaire, c'est juste une forme égale ou plus petite du mot. Cette approche est certainement la plus simple des deux approches.

Lemmatisation

La lemmatisation est le processus de recherche de la forme du mot existante dans le dictionnaire. C'est différent de Stemming. Cela implique des processus plus longs à calculer que la Stemming. La lemmatisation essaie de ramener un mot à sa forme racine avec une bonne manière.
La figure 2.2 présente les différence entre la stemming et la lemmatisation.

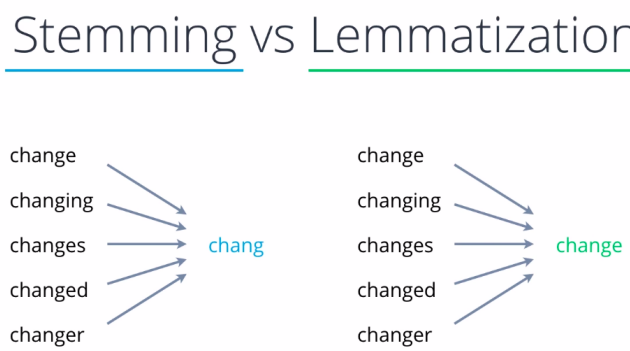


FIGURE 2.2 – Stemming et lemmatisation

2.4.3 Vectorisation du texte

Pour que le traitement du langage naturel fonctionne, il faut toujours transformer le langage naturel texte en forme numérique.

2.4.3.1 Bag of words (BoW)

Le BOW est un modèle simple utilisé pour produire des vecteurs numériques à partir du texte. Un texte défini comme une phrase ou un document, est représenté comme un vecteur dont la taille est égale au vocabulaire du corpus utilisé. Par conséquent, chaque phrase ou document est un sac de ses mots (BOW), ce qui signifie qu'un vecteur numérique le représente avec 1 à la position correspondante du vecteur et 0 au reste.
Cette approche ne tient compte ni l'ordre des mots ni l'information contextuelle présent dans la phrase ou le document contient.
On peut rassembler tous les mots W_i de notre vocabulaire pour former un sac de mots, $V = wi : i \in 1, ..., N$. Par exemple, si la séquence de texte est "J'adore le domaine de NLP", le vocabulaire devra inclure un index pour chacun des mots et des symboles. $V = Je, adore, le, domaine, de, NLP$.
À partir de là, la représentation vectorielle la plus simple pour distinguer les mots est celle des vecteurs monochromes qui indiquent à quel mot du vocabulaire le vecteur correspond.

Je=[100000]
$adore$=[010000]
le=[001000]‘
$domaine$=[000100]
de=[000010]
nlp=[000001]

Cependant, ces vecteurs simples ne sont pas une entrée très utile pour la plupart des tâches de traitement du langage naturel, car ils sont intégrés dans un espace vectoriel qui ne contient aucune information supplémentaire sur la signification contextuelle des mots représentés ni sur les relations entre eux.

2.4.3.2 TF-IDF « Term Frequency-inverse document frequency »

Le TF-IDF est une méthode simple utilisée pour produire des vecteurs numériques à partir du texte de manière similaire à l'approche du « Bag of words ». Le Tf-idf reflète l'importance d'un mot dans un document ou dans une collection de phrases. Elle est utilisée comme schéma de pondération des termes, qui augmente proportionnellement au nombre de fois où chaque mot apparaît dans un document et qui est normalisé par le nombre de documents ou de phrases dans le corpus.
Le poids de chaque mot w dans le document d est calculé comme suit :

$$q(w) = f_d(w) * log\frac{|D|}{f_D(w)} \tag{2.1}$$

où $f_d(w)$ est la fréquence de terme du mot w dans le document d, $f_D(w)$ est le nombre

de documents qui contiennent le mot w et $|D|$ est le nombre de documents dans la collection D.
Les techniques de vectoriser un texte, à savoir « Bag of words » et tf-idf, qui sont des choix très populaires pour les algorithmes d'apprentissage automatique traditionnels, peuvent aider à convertir du texte en vecteurs de caractéristiques numériques. Mais il existe des techniques plus avancées telles que :

2.4.3.3 Word2Vec

Word2Vec a été présenté pour la première fois dans [33]. Cette méthode permet de trouver une relation sémantique ou syntaxique entre les mots, ce qui n'était pas possible par l'approche traditionnelle TF-IDF. Elle a proposée deux nouvelles architectures, construites pour calculer les représentations vectorielles continues des mots. Ces deux architectures sont Skip-Gram et Continuos Bag of Words (CBoW). Les deux architectures reposent sur un réseau neuronal FeedForward, qui a été introduit pour la première fois dans [12].

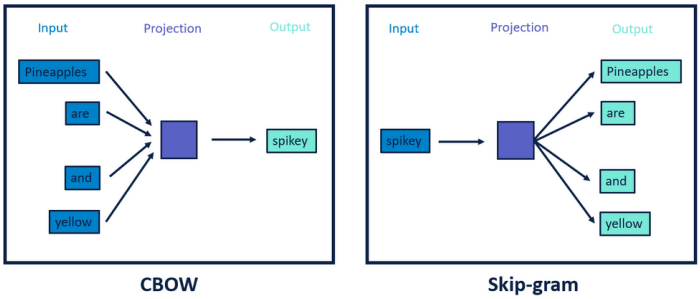


FIGURE 2.3 – Architecture de l'approche CBOW et Skip-Gram [7]

Continuous Bag of Words (CBoW) :

L'architecture du Continuous Bag of Words (CBoW) est similaire au réseau neuronal feedforward. Cependant, la couche de projection est partagée pour tous les mots. Cette architecture prédit le mot cible pour un ensemble donné de mots de contexte dans la taille de la fenêtre (utilisation du contexte pour prédire un mot cible).

Modèle Skip-Gram :
L'architecture Skip-Gram est similaire à l'architecture Continuous Bag of Words (CBoW). Cette architecture prédit les mots de contexte pour une taille de fenêtre donnée avec le mot central donné[26].
L'objectif du skip-gram, contrairement au CBOW, est de prédire le contexte d'un mot, donc pour chaque contexte de mot, il y a une valeur cible et une sortie spécifique c'est à dire pour un mot d'entrée, on a plusieurs mots de contexte de sortie.

Dans les sections précédentes, on a présenté les méthodes linguistiques de NLP, les parties suivantes seront consacrées aux méthodes d'apprentissage automatique, qui porte sur l'application de modèles de machine Learning et Deep Learning.

2.5 L'apprentissage automatique

L'apprentissage automatique est un sous-ensemble de l'intelligence artificielle associé à la création d'algorithmes qui peuvent se changer sans intervention humaine pour obtenir le résultat souhaité.
Les paragraphes suivants présentent certains classifieurs de texte utilisés.

2.5.1 SVM

La machine à vecteurs de support (SVM)[8] est un algorithme d'apprentissage automatique supervisé. Chaque donnée est représenté par un point dans un espace à n dimensions. Son objectif est de trouver un hyperplan qui sépare les points de données en des classes différentes. Il peut y avoir de nombreux hyperplans possibles qui peuvent séparer un tel ensemble de points de données, mais le but est de choisir le meilleur. Cela se traduit par la recherche du plan qui a la marge maximale, c'est-à-dire la distance maximale entre les points de données de différentes classes. Cela renforce la conviction que les futurs points de données seront classés avec une meilleure certitude.

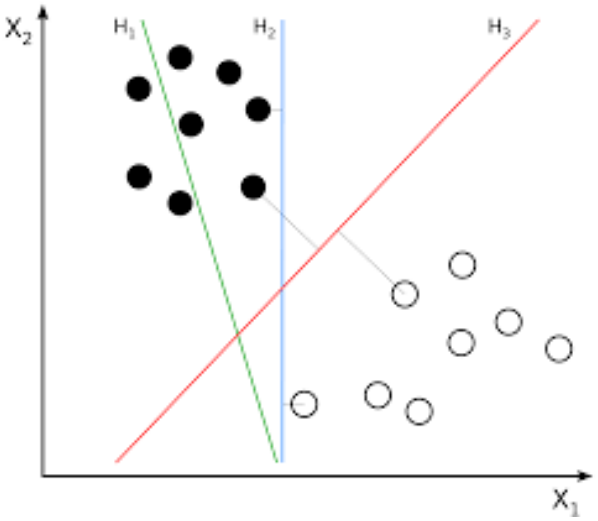


FIGURE 2.4 – Séparation des données à l'aide d'un hyperplan
[8]

La figure 2.4 met en évidence la séparation des données à l'aide d'un hyperplan, ce dernier permet de diviser l'espace en deux régions.

Différentes fonctions du noyau

Les algorithmes SVM utilisent un ensemble de fonctions mathématiques définies comme le noyau (Kernel). La fonction du noyau permet de prendre des données en entrée et de les transformer dans la forme requise, parmi ces fonctions, on peut citer :

Noyau linéaire :

$$z = yx + b$$

Noyau polynomial :

$$z = (yx + b)^k$$

Avec :
k : degré du noyau RBF et b : constant.

Noyau RBF :

$$z = exp(-\gamma||x - y||^2)$$

Avec : $||x—y||$: est la distance euclidienne entre x et y.
γ : est le paramètre de noyau RBF.

2.5.2 Naive Bayes

Le classifieur Naive Bayes est couramment utilisé dans les applications de classification de textes telles que l'analyse des sentiments (bon, mauvais), et également applicable aux problèmes de classification multi-classes. C'est un classifieur linéaire basé sur le théorème de Bayes, qui est le suivant :

$$P(j/i) = \left(\frac{P(i/j)P(j)}{P(i)}\right) \tag{2.2}$$

Sachant que i et j sont deux événements et $P(i) \neq 0$.

2.5.3 Random forest

Random forest est un algorithme d'apprentissage supervisé. Il peut être utilisé à la fois pour la classification et la régression. C'est aussi l'algorithme le plus flexible et le plus facile à utiliser. C'est un ensemble d'arbres de décision à partir d'un sous-ensemble sélectionné au hasard de l'ensemble d'apprentissage. Il agrège les votes de différents arbres de décision pour décider. On dit que plus il y a d'arbres, plus une forêt est robuste.
La figure 2.5 décrit l'architecture du modèle random forest.

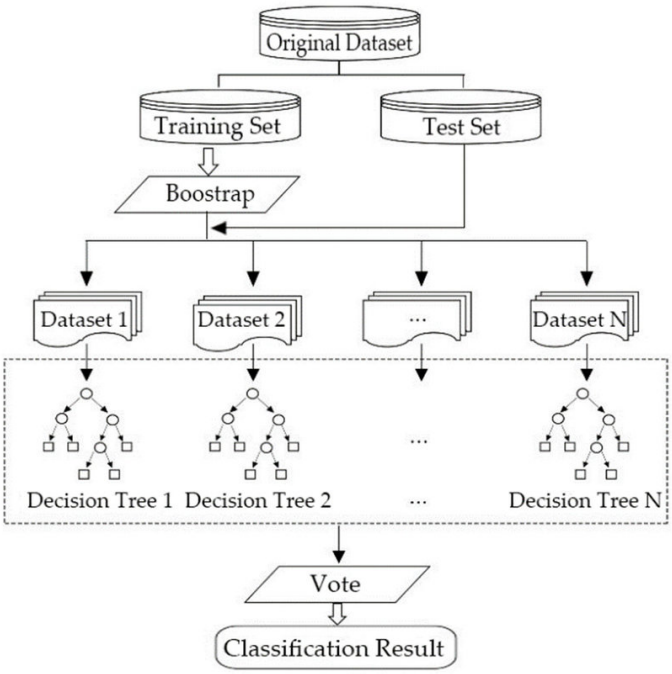


FIGURE 2.5 – Architecture Random Forest
[5]

2.5.4 Logistic regression

La régression logistique consiste à trouver une relation entre les caractéristiques et la probabilité d'un résultat particulier. La régression logistique multinomiale, est considéré comme un classifieur linéaire généralisé, car son résultat dépend de la somme de ses entrées et ses paramètres[28]. L'objectif est d'apprendre, à partir de l'ensemble d'apprentissage, la fonction de mappage f qui mappe une entrée X, qui est un vecteur de caractéristiques, à une classe Ci. Le classifieur y parvient en apprenant d'abord à partir de l'ensemble d'apprentissage un vecteur de poids, w et un biais, b. Ceci peut être représenté par une variable z.

$$z = wX + b \tag{2.3}$$

Où chaque nombre du vecteur w correspond à une caractéristique de l'échantillon d'entrée X, et représente l'importance de cette caractéristique dans la décision de classification. Le biais b décale la fonction pour s'assurer qu'elle ne doit pas nécessairement couper l'axe des y à l'origine, car les données ne sont pas nécessairement séparables par une fonction qui passe par l'origine.

2.6 L'apprentissage profond

L'apprentissage profond ou (deep learning en anglais) est un sous-domaine de l'apprentissage automatique concerné par des algorithmes inspirés de la structure et de la fonction du cerveau appelés réseaux de neurones artificiels.

On va présenter les différents composants de l'architecture d'un réseau de neurones telles que les couches (layers), ainsi que quelques concepts de base des réseaux de neurones comme la fonction d'activation, la fonction de perte, etc.

2.6.1 Layer

Une couche dans un modèle d'apprentissage en profondeur est une structure ou une topologie de réseau dans l'architecture du modèle, qui prend les informations des couches précédentes, puis les transmet à la couche suivante.

2.6.2 Forward propagation et back propagation

La mise à jour des poids dans le réseau de neurones nécessite un mécanisme de propagation vers les neurones de chaque couche. Ce processus est décomposé en deux parties : "forward propagation" et "back propagation".

Forward propagation :

L'unité d'entrée fournit l'information initiale qui se propage ensuite aux neurones de chaque couche cachée (l'information se déplace dans une seule direction, vers l'avant) et enfin elle produit un poids en sortie.

Back propagation :

Le back propagation est un processus visant à réduire la fonction de perte entre la sortie attendue et celle prédite en mettant à jour les paramètres (les poids) du réseau de neurones.

2.6.3 Fonction d'activation

Une fonction d'activation est une caractéristique très importante d'un réseau de neurones artificiels. Elle décide essentiellement si le neurone doit être activé ou non.

Fonction Softmax :

La fonction softmax est utilisée dans diverses méthodes de classification multi-classes. Elle est une généralisation de la fonction logistique (fonction sigmoïde). La fonction logistique prend un nombre réel x en entrée et renvoie un nombre réel en sortie. Le nombre réel renvoyé se situe dans l'intervalle $(0, 1)$.
La fonction Softmax prend un vecteur de x en entrée et renvoie un vecteur z de nombres réels en sortie. L'entrée dans z est dans un rang de $[0, 1]$ et la somme de toutes les entrées est égale à 1.

Fonction Relu :

RELU est la fonction d'activation la plus utilisée, la sortie est égale à l'entrée pour toutes les valeurs positives et zéro pour toutes les valeurs négatives.
Mathématiquement, elle est définie comme :

$$y = max(0, x)$$

2.6.4 Fonctions de perte

La fonction de perte représente la pénalité générée lorsque l'estimation de la cible fournie par le modèle n'est pas parfaitement égale à la cible. Donc on cherche a minimiser la valeur de perte.

Parmi les fonctions de perte, on peut citer :

La perte d'entropie croisée catégorielle, définie comme suit :

$$L = -\sum_{j=1}^{M} y_{i,j} log(p_{i,j})$$

Avec : M est le nombre de classes, y est l'indicateur binaire (0 ou 1), P est la probabilité prédite que l'observation i soit égal à la classe j.
Dans le cas de la classification binaire, la fonction de perte d'entropie croisée binaire devient :

$$L = -(ylog(p) + (1 - y)log(1 - p))$$

Après avoir présenté les différentes caractéristiques d'un modèle DL, on va décrire dans

les sections suivantes, les principaux modèles de deep learning de NLP.

2.6.5 Multi-layer perceptrons

Les outils et les algorithmes actuels d'apprentissage profond sont nés de la tentative de modélisation mathématique des neurones du cerveau dans les années 50. Ces premiers apprenants Perceptron, introduits par Rosenblatt, ne pouvaient modéliser que des frontières de décision simples. Des développements ultérieurs ont introduit des capacités plus complexes rendues possibles par le perceptron multicouche (MLP)[11]. L'extension du perceptron simple à une variante multicouche a permis de résoudre des problèmes plus complexes, car les couches cachées multiples permettent de modéliser des limites de décision non linéaires. L'une des grandes évolutions entre le perceptron et sa variante multicouche est la manière dont il est entraîné. Il n'est pas possible d'utiliser une fonction échelon pour entraîner les MLP, il a donc fallu introduire de nouvelles fonctions d'activation.

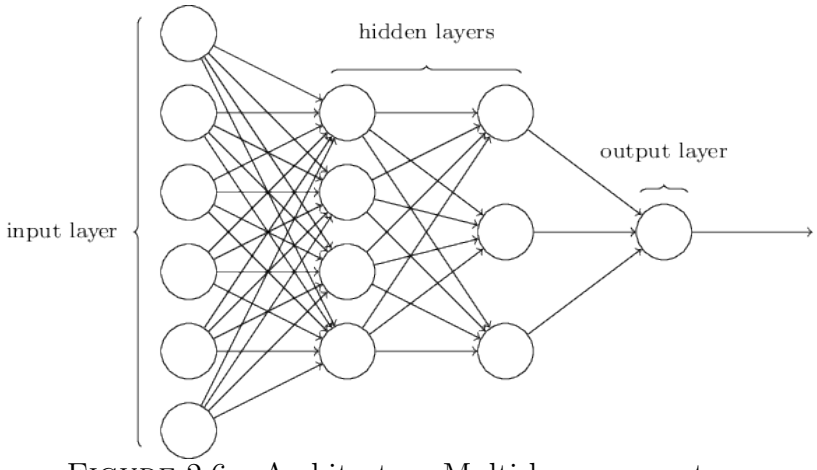


FIGURE 2.6 – Architecture Multi-layer perceptrons [5]

2.6.6 RNN

Les réseaux neuronaux récurrents (RNN) sont une extension des MLP qui permet de résoudre certains de leurs problèmes temporels. En NLP, la manipulation de séquences de données pose des problèmes particuliers aux MLP, car ils n'ont aucun moyen de stocker l'état précédent dans une séquence. De plus, les textes sont rarement normalisés en longueur et ne peuvent pas être divisés facilement tout en gardant l'information originale intacte. Le RNN résout ce problème en introduisant un état interne (hidden state) qui est mis à jour en fonction de l'état actuel et du prochain symbole d'entrée. Les symboles de sortie sont générés en fonction de l'état interne. Les RNN rendent cela possible en introduisant la notion de temps dans le système. Ce qui permet aux

RNN de traiter des séquences d'entrées, où les jetons d'entrée peuvent être interprétés différemment en fonction des autres jetons[29].

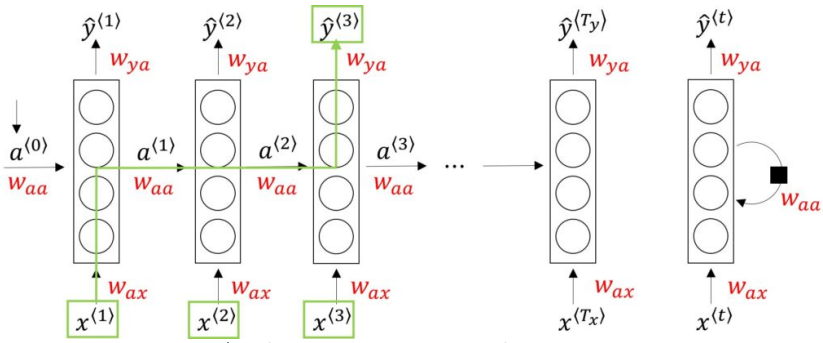


FIGURE 2.7 – Architecture réseau de neurones récurrents.
[9]

2.6.7 LSTM

LSTM[30], qui sont un type particulier de RNN, ont été conçus pour traiter le problème des dépendances à long terme. Les réseaux LSTM, bien que semblables aux RNN, diffèrent dans la manière dont ils traitent l'information. Ils apportent des modifications mineures à l'information en utilisant des multiplications et des additions. Cela permet aux LSTM de se souvenir ou d'oublier sélectivement des informations sur de longues périodes.

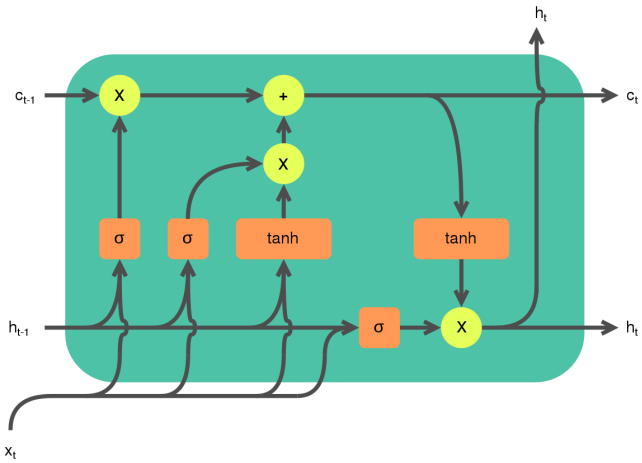


FIGURE 2.8 – Illustration de LSTM.
[1]

Avec :

X_t est le vecteur d'entrée.
C_t est la mémoire du bloc courant.
$C_t - 1$ est la mémoire du bloc précédent.
$h_t - 1$ est la sortie du bloc précédent.
h_t est la sortie du bloc courant.
$Tanh$ est la tangente hyperbolique.
γ est la fonction sigmoïde.
$+$ est une addition élément par élément.
X est la multiplication élément par élément.

L'une des complexités de le NLP est que le texte peut avoir des dépendances futures aussi bien que des dépendances passées, Ceci est dû à l'apparition de modèle Bidirectional-LSTM.

2.6.8 Bidirectional-LSTM

Bidirectional-LSTM est une version améliorée du LSTM où, un NN est formé en considérant le texte de gauche à droite et de droite à gauche, de sorte que la sortie fonctionne comme une concaténation des deux couches.
La figure 3.8 présente l'architecture du modèle BILSTM.

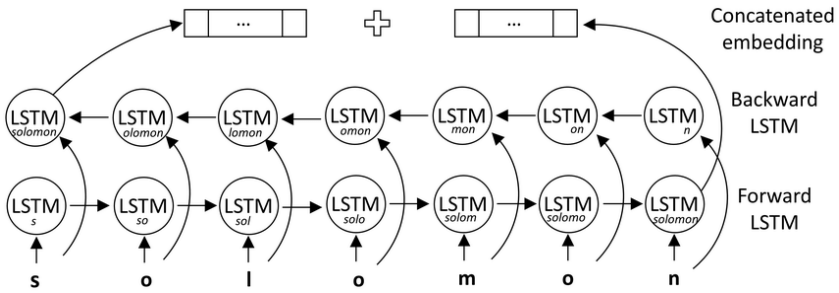


FIGURE 2.9 – Illustration de BILSTM.
[3]

2.7 Apprentissage par transfert

Nous, les humains, avons la capacité inhérente de transférer nos connaissances entre différentes tâches. On utilise les connaissances acquises lors d'une tâche pour résoudre d'autres tâches similaires. Plus les tâches sont pertinentes, plus il nous est facile de transférer ou d'utiliser nos connaissances de manière croisée, c'est exactement le même principe qui s'applique à l'apprentissage par transfert. L'apprentissage par transfert ou Transfer Learning en anglais désigne l'ensemble des méthodes qui permettent de transférer les connaissances acquises à partir de la résolution d'un problème donné pour traiter un autre problème. La figure 3.3 présente une illustration du processus d'apprentissage par transfert.

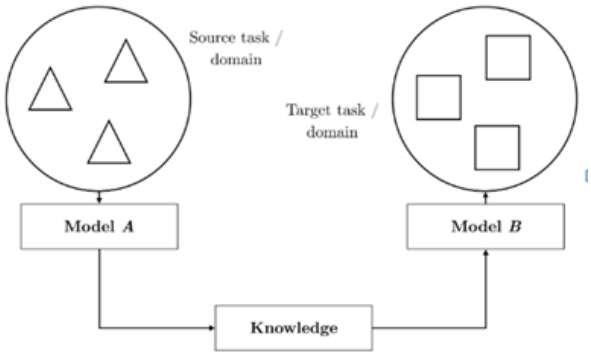


FIGURE 2.10 – Illustration du processus d'apprentissage par transfert

2.8 Transformers

Le "Transformers" est un réseau neuronal présenté pour la première fois dans[32], qui dépend entièrement du mécanisme d'attention mentionné dans[32], permet d'obtenir des résultats de « state of the art » dans de nombreuses tâches de NLP. «Attention is all you need »[32] a été le premier à introduire une nouvelle ère dans la tâche de traitement du langage naturel appelée modélisation du langage. Il améliore les architectures ENCODER-DECODER.

La figure 2.11 illustre l'architecture du "transformers" telle qu'elle a été présentée dans [32]

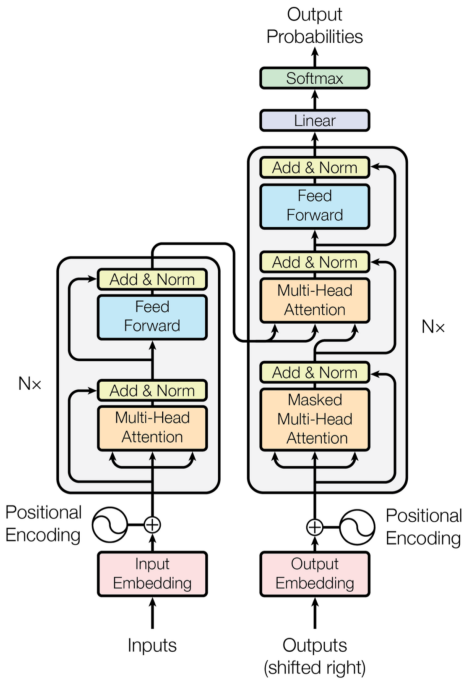


FIGURE 2.11 – Architecture du transformers

2.8.1 BERT

Malgré la facilité et la flexibilité de l'utilisation des mots (embedding) pré-formés dans la modélisation du langage, ces mots présentent un inconvénient majeur chaque mot est représenté par un seul vecteur. Cela signifie qu'un mot qui a plusieurs significations dans différents contextes, n'est intégré que dans une partie de l'espace ce qui entraîne une perte d'informations linguistiques. Ce problème est théoriquement résolu en utilisant des RNN ou des LSTM qui peuvent transporter des informations de façon continue sur des séquences pour créer un contexte pour le mot actuel. Cependant, en pratique ces modèles peuvent s'avérer insuffisants pour fournir des données contextuelles riches pour prédire des caractéristiques linguistiques lourdes en contexte, comme l'humour ou le sarcasme.
Il peut donc être difficile pour un modèle de savoir quelles caractéristiques transmettre au cours de la formation. Ce problème de contexte linguistique des RNN a été résolu avec des modèles "Bidirectional-LSTM"[16] et des mécanismes d'attention avec un certain succès.
Une nouvelle méthode pour fournir des informations riches en contexte au réseau est l'architecture BERT de Google[16] basée sur les "transformers"[32]. Au début d'octobre, 2018, Google a publié son modèle NLP appelé "Bidirectional Encoder Representations from Transformers (BERT)"[13]. Les auteurs montrent qu'il est capable de marquer des résultats de pointe (state of the art) pour onze tâches de NLP. Les "transformers" sont basés sur des principes totalement différents de modèles neuronaux récurrents de langage, ils s'appuient sur l'attention pour ses représentations au lieu de la récurrence.
L'objectif d'implémenter BERT était de créer un système de codage à partir de "transformers". Un modèle capable de prendre en compte le contexte complet d'un jeton à gauche et à droite. Afin d'utiliser BERT pour une tâche spécifique deux étapes sont nécessaires :

- Le modèle doit être pré-entraîné. Cela signifie que le modèle n'est pas encore formé de manière spécifique à une tâche, mais qu'il est plutôt formé pour coder le langage lui-même d'une manière raisonnable. Cela permet au même modèle d'être utilisé pour plusieurs tâches différentes. Le pré-entraînement aboutit à un "transformers" d'usage général qui peut coder les mots d'entrée.

- Une fois que le modèle est initialisé avec les paramètres obtenus par le pré-entraînement. Il peut être affiné pour être utilisé pour une tâche particulière. Pour ce faire, il n'est pas nécessaire de modifier l'architecture du modèle lui-même. Au contraire le même modèle pré-entraîné peut être appliqué à plusieurs tâches différentes en superposant des couches spécifiques aux tâches et en entraînant le modèle sur des données d'entraînement étiquetées relatives à la tâche souhaitée, c'est "le Fine-Tuning".

La figure 2.12 présente l'architecture du BERT

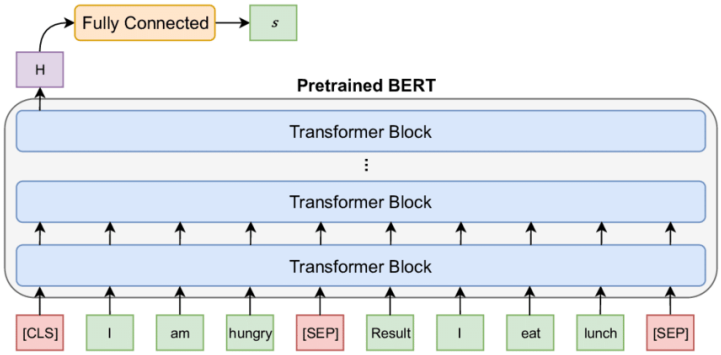


FIGURE 2.12 – Architecture BERT

BERT est composé d'une couche d'embedding suivie par 12 couches (ENCODER) qui n'ont pas les mêmes matrices de poids : une dimension de la couche cachée de 768 et 12 tête d'attentions.[13]

La couche embedding : où chaque mot est représenté par un vecteur de dimension 512.

Chaque ENCODER est composé principalement par deux sous-couches :

Un bloc d'attention : applique un mécanisme d'attention spécifique appelé "self attention". Ce dernier permet aux modèles d'associer chaque mot de l'entrée à d'autres mots.

Une layer Feed forward : c'est une simple Fully connected layer.

2.8.2 CamemBERT

Fin octobre 2019, une équipe de Facebook AI Research, Inria et Sorbonne Université ont publié CamemBERT, essentiellement une version française du modèle de langage pré-entraîné révolutionnaire de Google AI BERT.
Les modèles de langage pré-entraînés avec des architectures basées sur les "transformers" [32] telles que BERT, GPT-2, RoBERTa, ALBERT et T5 ont permis des progrès rapides dans la science de le NLP et de la compréhension du langage naturel (NLU), notamment en modifiant la façon dont les machines traitent le langage naturel.
Cependant, la plupart des meilleurs modèles issus de ce domaine de recherche ont été formés et dirigés vers la langue anglaise.
RoBERTa[21] par exemple a été formé sur plus de 100 Go de données en anglais. Étant donné que les performances des modèles linguistiques pré-entraînés peuvent être considérablement améliorées en utilisant davantage de données d'entraînement, les chercheurs ont collecté donc une quantité énorme des données textuelles en français à partir du grand corpus multilingue OSCAR nouvellement disponible. Le corpus français OSCAR

comprend 138 Go de texte non compressé et 32,7 milliards de jetons SentencePiece.[20] Les chercheurs ont évalué CamemBERT sur quatre tâches en aval :

- Part-of-speech (POS) tagging : une tâche syntaxique qui consiste à attribuer chaque mot à sa catégorie grammaticale correspondante.
- Dependency parsing, qui consiste à prédire l'arbre syntaxique étiqueté en capturant les relations syntaxiques entre les mots
- Named Entity Recognition (NER) : une tâche d'étiquetage de séquence pour prédire quels mots font référence à des objets du monde réel, tels que des personnes, des lieux et des organisations.
- Natural language inference (NLI) : est la tâche de déterminer la relation d'inférence entre deux textes : implication, contradiction ou neutre.

CamemBERT[22] a surpassé les autres modèles francophones dans les tests. Les chercheurs pensent que le nouveau modèle pré-entraîné peut être efficacement affiné pour diverses tâches en aval et qu'il ouvre une voie prometteuse pour la recherche future en NLP français.

2.9 Conclusion

Dans ce chapitre, on a exploré l'état de l'art de notre projet. On a commencé par une définition des concepts théoriques utilisés pour le traitement de langage naturel. Puis, on a étudié les modèles de langues existants, leurs fonctionnalités et leurs limites.
Le chapitre suivant sera consacré à une description des différents modèles qui ont été implémentés au cours de ce projet.

Chapitre 3

Prétraitement des Données et Modélisation

3.1 Introduction

Le but de ce projet est de rechercher et d'incorporer les dernières techniques dans la NLP et de les comparer avec les techniques traditionnelles.
Ce chapitre est composé de deux sections. Dans une première section, on va se concentrer sur la conception des modèles de classification d'intention. Dans la deuxième section, on va présenter la conception des modèles de l'analyse sentimentale, et enfin on va présenter brièvement l'environnement de développement.

3.2 Classification d'intention de l'utilisateur

Tout d'abord, on commence par la description des différents processus de réalisation des modèles de classification d'intention.

3.2.1 Description de l'ensemble de données

Afin de créer notre dataset, il existe un certain nombre de ressources internes principalement des e-mails entre les agents de la compagnie et les clients.

Notre base des données est composée par environ 3000 conversations. Avant de commencer la partie de nettoyage de données, on a implémenté un script python que permet de séparer les questions et les réponses en deux colonnes différentes.
Puisque généralement Globalnet ayant des réponses standard donc on a choisi d'implémenter un chatbot basé sur la classification d'intention.

En considérant que la structure du courrier n'est pas fixe tout dépend de la source des e-mails ainsi une conversation peut contenir un ou plusieurs questions de client.

3.2.2 Étiquetage de données

L'étiquetage des données est le processus d'identification des données brutes (images, fichiers texte, vidéos, etc.) et d'ajout d'une ou plusieurs étiquettes significatives et informatives pour fournir un contexte afin qu'un modèle d'apprentissage automatique puisse apprendre. Donc c'est une étape fondamentale pour notre projet pour créer un modèle performant et des résultats acceptables. Pour cela les données doivent être étiquetées correctement.

3.2.2.1 Similarité cosinus

•La similarité cosinus : est une métrique utilisée pour mesurer la similitude entre deux phrases. Cette technique permet de calculer la similitude en mesurant le cosinus de l'angle entre deux vecteurs. Elle donne une valeur comprise entre -1 et 1, et pour les vecteurs positifs donne une valeur entre 0 et 1. Si deux vecteurs sont semblables, leur similarité cosinus doit être égal à 1, et s'ils sont très différents, il doit tendre vers -1. L'équation suivante illustre la formule de similarité cosinus.

$$\sum_{i=1}^{n} \frac{(A_i B_i)}{\sqrt{\sum_{i=1}^{n}(A_i)^2}\sqrt{\sum_{i=1}^{n}(B_i)^2}} \tag{3.1}$$

Pour étiqueter notre base d'apprentissage, on commence d'abord par la préparation d'un fichier csv composé par les e-mails standard de Globalnet et associe à chacun un contexte.

Pour que notre solution soit efficace, on doit supprimer les mots formels et fréquemment utilisés dans tous les e-mails de Globalnet comme "cordialement","bonjour","cher", "client" pour éliminer le bruit. C'est-à-dire pour mesurer la similarité, on a utilisé uniquement le contenu principal des e-mails.

Les réponses de Globalnet sont standards, cette information nous aide dans la tâche d'étiquetage des données.
Afin d'étiqueter les questions de client, on a utilisé la cousine similarité entre la réponse fournie par l'agent au client et les réponses standards. Donc l'idée est d'associer pour chaque question d'utilisateur le contexte de réponse standard avec la valeur de la similarité cousins la plus élevée.

3.2.3 Préparation et pré-traitement des données

Une grande partie du travail consiste à bien nettoyer les données. Le pré-traitement fournira un texte nettoyé qui peut être compris par la machine. Pour ce faire, il existe de nombreuses méthodes et techniques proposées et bien documentées pour le pré-traitement de texte afin d'automatiser ce processus.

La figure 3.1 illustre les différentes étapes de nettoyage de données.

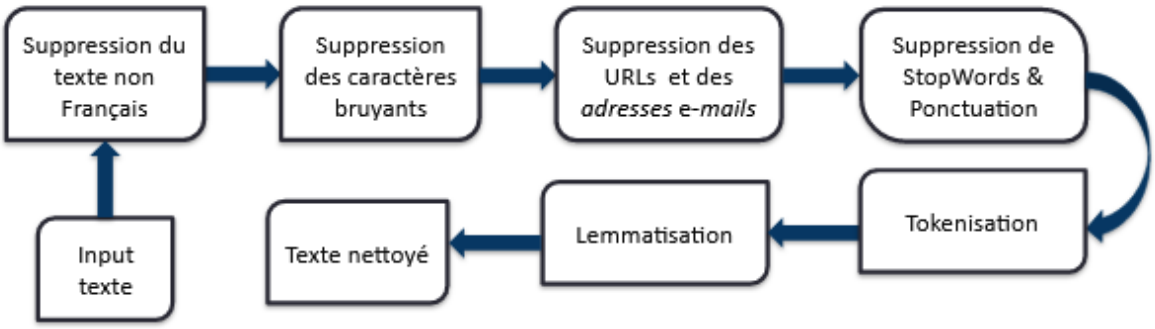


FIGURE 3.1 – Pipeline de nettoyage de données.

Dans le cas des e-mails, les données sont très bruyantes et nécessitent une attention particulière définie par des étapes de nettoyage initiales qui sont résumées dans la figure 3.1

- **Suppression du texte non français** :

  Puisque notre modèle dépend d'une seule langue (le français), les autres langues de données textuelles doivent être considérées comme du bruit (dialecte tunisien, anglais , arabe et italien). On doit être sûr que les données de texte d'entrée de mon modèle sont tous en français. Une incohérence dans les données entraînera une diminution de la précision du modèle, même les autres langues ne représentent qu'une petite partie de l'ensemble des données disponible ainsi les API gratuites pour la traduction imposent des contraintes sur la qualité d'emails traduits. on doit donc détecter le texte non français et le supprimer de notre base d'apprentissage grâce à la bibliothèque spacy-langdetect[31]

- **Suppression des caractères bruyants** :

  Pour coder les textes, on a utilisé le format de l'encodage "utf-8". C'est l'un des encodages les plus utilisés dans le monde numérique. Puisque les mots peuvent se trouver à plusieurs endroits dans les phrases, il est nécessaire ainsi de mettre le texte en minuscule.

- **Suppression des Urls et des emails** :

  Ils ne contient pas d'opinion d'utilisateur, c'est pourquoi on doit le supprimer.

- **Suppression StopWords et les points de ponctuations** :

  Les stopwords qui sont un ensemble de mots couramment utilisés dans une langue, tels que « et », « un », « le », contiennent très peu d'informations et apparaissent fréquemment. Le problème est donc que les modèles peuvent se concentrer sur les mots importants si ces Stopwords sont supprimés du texte et c'est le même cas pour la ponctuation.

- **Tokenisation** :

  La tokenisation du texte en petits morceaux "jetons". C'est la base de toutes les tâches de haut niveau puisqu'elle définit l'entité sur laquelle les modèles essaient d'apprendre.

  Pour utiliser le modèle BERT/CamemBERT pré-entraîné, on doit convertir les données d'entrée dans un format approprié pour que chaque phrase puisse être envoyée au modèle. l'utilisation de BERTTokenizer et CamemBERTTokenizer est basé respectivement sur les algorithmes "wordpiece"[34] et "sentencepiece"[20].

- **Lemmatisation** :

  Dans certains cas, les mots peuvent apparaître sous plusieurs formes : "adjectif","verbe","nom","adverbe",etc. Donc il est nécessaire d'utiliser la racine des mots afin d'éviter le bruit dû aux nombreuses formes qui revient au même mot.
  Les différents motifs sont mis en correspondance à l'aide d'expressions régulières, ce qui permet un processus de nettoyage automatique.
  Voici un exemple d'un e-mail avant le processus de nettoyage dans la Figure.3.2

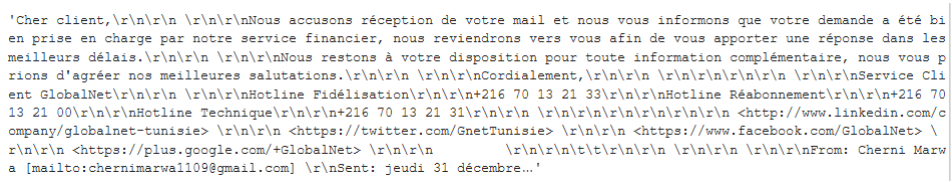


FIGURE 3.2 – Un e-mail avant pré-traitement

Les figures 3.3 et 3.4 illustrent les données avant et après le processus de nettoyage de données.

	Sujet	Column1	Column2	Column3	Column4	Column5	Column6	Column7	Column8	Unnamed: 9
0	NaN	Message client : doublement de bebi	NaN	Réponse mail : Suite à votre e	mail du 23/01/2019 et afin de bénéficier de 8M...	NaN	NaN	NaN	NaN	NaN
1	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
2	NaN	Message client : Ca fait plus qu un mois que ...	NaN	Réponse mail : Suite à votre e	mail du 02/02/2019 et afin de vous aider à dia...	vous que vous n'avez pas de bruit parasitaire ...	Fi, • Il est important de tester votre connexi...	NaN	NaN	NaN
3	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
4	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN

FIGURE 3.3 – Les données avant la phase de prétraitement

	Questions	Reponses
0	merci de votre réponse mais moi je suis censé...	nous tenons à vous informer que la durée de v...
2	je suis abonné avec vous sous le code code con...	nous avons le regret de vous informer que vot...
3	vje veut connaitre le procedere de isolation ...	nous accusons réception de votre mail et nous...
5	bonjour, quelle est la démarche à suivre pour ...	nous accusons réception de votre mail et nous...

FIGURE 3.4 – La base d'apprentissage après la phase de prétraitement

Selon les réponses standards de Globalnet, on a obtenu 396 intentions d'utilisateur.

Après la collecte et le pré-traitement des données, on est resté avec des classes déséquilibrées, c'est a dire la différence entre les nombres des séquences de texte pour chaque intention d'utilisateur est très élevée.

On supprime les classes les moins demandées par les clients. Ensuite, on essaye de combiner les classes minoritaires et les plus similaires en une seule classe. Le nombre des intentions a été diminué mais le déséquilibre n'est pas résolu totalement.

Alors regrouper les classes selon un contexte plus général, apparaît comme la

meilleure solution. Cette approche va diminuer le bruit, le temps d'exécution et elle va éviter partiellement le problème de classes fortement déséquilibrées.
Donc on a regroupé les questions en 4 groupes traitant quatre termes principaux : technique, information générale, commerciale et financier. Au lieu de faire un seul modèle de classification d'intention, on a implémenté deux modèles séquentiels : modèle de classification contextuel suivi par un modèle de classification d'intention.

	Questions	sousclasse_v2	classpr
307	merci pour votre rapide réponse. toutefois j...	changer modem	Demandec_commerciale
308	j'ai eu plusieurs problème a la connexion d'...	changer modem	Demandec_commerciale
309	la connexion internet flanche ça vient ça re...	changer modem	Demandec_commerciale
310	il y a un probleme recent avec ma connexion ...	changer modem	Demandec_commerciale
311	j'ai constaté que le modem actuel à ma posse...	changer modem	Demandec_commerciale
312	j'ai un problème au niveau du modem. enfaite...	changer modem	Demandec_commerciale
313	j'ai fait beaucoup de réclamation pour le ch...	changer modem	Demandec_commerciale
314	bonsoirj'ai un problème au niveau du modem. e...	changer modem	Demandec_commerciale
315	j'ai fait beaucoup de réclamation pour le cha...	changer modem	Demandec_commerciale
316	j'ai un abonnement avec gnet forca 8mo qui va...	migration de forca vers uno	Demandec_commerciale

FIGURE 3.5 – Données collectées

3.2.4 Augmentation des données

La quantité de données, dont on dispose pour entraîner notre modèle a un impact considérable sur les performances du modèle. Malgré toutes les approches qu'on a utilisé pour résoudre le problème de la rareté des données, ce dernier n'était pas résolu définitivement. Donc grâce à la bibliothèque nlpaug[14], on a essayé avec quelques techniques d'augmentation de données en NLP, ces dernières peuvent être utilisées pour augmenter artificiellement la taille d'un ensemble de données d'apprentissage en générant des versions modifiées de textes dans l'ensemble de données. [6]

NLPAug propose trois méthodes d'augmentation, on peut citer :

1. **Thesaurus** :

   L'un des moyens les plus utiles d'augmenter le texte consiste à remplacer des mots ou des phrases par leurs synonymes. C'est une technique proposée par Zhang et al[15]. Elle permet de sélectionner un mot et de remplacer par des synonymes selon une distribution géométrique.

2. **Word embedding** :

   Wang et Yang ont proposé dans[33] d'utiliser le système KNN (k-nearest-neighbor) et la similarité cosinus pour trouver le mot similaire à remplacer. On a la possibilité d'utiliser le word embedding classique pré-entraîné tel que word2vec, Glove et fasttext pour effectuer une recherche de similarité. Pour notre cas on a utilisé word2vec.

3. **Contextualized word embeddings** :

   Au lieu d'utiliser des embeddings de mots statiques, une autre approche a été proposée dans [25] qu'utilise des words embedding contextualisés pour remplacer le mot cible. Dans ce sens, Kobayashi propose d'implémenter un modèle de langage bidirectionnel Contextual[19] pour générer plus des données. Ce modèle a été entraîné en utilisant la technique de "Modélisation du langage masqué" où le modèle doit prédire des mots masqués en fonction du contexte. Cette tâche est utilisée pour compléter certains textes. c'est à dire on peut masquer un mot du texte et demander au modèle de prédire le token masqué.

Après avoir testé plusieurs techniques, on a trouvé que la méthode la plus appropriée pour notre base d'apprentissage est la dernière technique en utilisant l'architecture du modèle CamemBERT[22]. Ceci est attendu puisque CamemBERT a été entraîné sur une quantité énorme de données françaises.
Exemple d'augmentation de données textuelles.

•Texte originale :

J'ai eu un problème de connexion internet.

•Nouvelles séquences :

J' iciai eu un problème particulier de connexion internet.
Bonsoir J'ai eu un problème de mauvaise connexion internet.
J'ai eu également un problème de de connexion internet.
J'ai eu un sérieuse problème de communication connexion internet.

La répartition des séquences textuelles pour chaque classe avant l'augmentation de données est représenté dans la figure 3.6.



FIGURE 3.6 – Des classes déséquilibres

Après l'augmentation de données, on a rencontré un autre problème avant de commencer la partie modélisation.
Ce problème est apparu, puisque il existe des types de questions que le client ne les pose pas généralement par un e-mail comme, demande les nouvelles promotions, l'horaire, etc.

La solution consistait à générer des nouveaux donnés par nous-mêmes en utilisant un outil "Chatito[27] qui permet de générer des ensembles de données pour la formation et la validation de modèles de chatbots à l'aide d'un simple DSL.

L'idée derrière cet outil est d'avoir une intersection entre l'augmentation de données et une description des combinaisons de phrases possibles. Il n'est pas destiné à générer des ensembles de données déterministes qui peuvent dépasser un modèle de phrase unique. Dans ce cas, on doit avoir un certain contrôle sur les chemins de génération et ne tirer que les échantillons nécessaires.

3.2.4.1 Pré-traitement BERT

Puisque on a traité des textes français, on a choisi la version multilangue de BERT (Multilingual) qui supporte 104 langues au lieu d'utiliser BERT.
Les embeddings mBERT sont formés avec deux méthodes de formation :

1. Tâche de classification : Déterminer dans quelle catégorie la phrase d'entrée doit être classée.
2. Prédiction de la phrase suivante : Détermine si la deuxième phrase suit naturellement la première.

Pour une tâche de classification, une séquence doit être passée par les étapes suivantes avant d'être introduite dans le modèle BERT :
•Tokenisation : décomposition de la phrase en jetons.
•Ajout du jeton $[CLS]$ au début de la phrase.
•Ajout du jeton $[SEP]$ à la fin de la phrase.
•Ajout des jetons $[PAD]$ pour que la longueur totale de phrase soit égale à la

longueur maximale.
•Convertir chaque token en son ID correspondant dans le modèle.

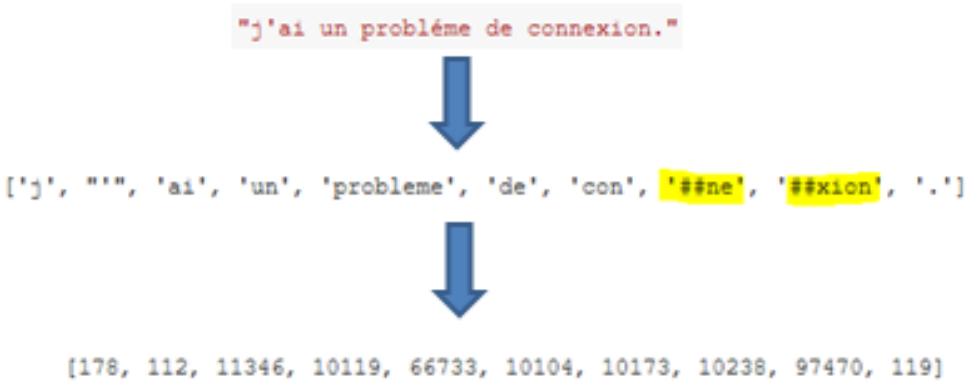


FIGURE 3.7 – Exemple de pré-traitement de texte en utilisant mBERT.

L'idée derrière BERT Tokenizer est de prendre chaque mot du texte et de le convertir en embedding. Puisque BERT est un modèle pré-entraîné, on doit en quelque sorte utiliser son vocabulaire.
Mais comment BERT gère-t-il le vocabulaire qui n'est pas dans sa collection de tokens et qu'il existe dans notre base d'apprentissage ? (le cas de mot connexion dans la figure 3.7.)

Une des techniques les plus anciennes pour gérer cela est de définir un token inconnu et d'avoir un embedding pour ce token, ce qui n'aide pas vraiment. BERT fait quelque chose un peu plus intelligent, lorsqu'il ne trouve pas le mot embedding dans son vocabulaire, il utilise un modèle de mot-clé et il décompose le mot embedding en plusieurs sous mots. Par exemple BERT n'avait pas un embedding pour le mot "connexion" mais il avait pour "con" un autre pour "ne" enfin "xion".

3.2.4.2 Prétraitement CamemBERT

La tokenisation pour camemBERT est basé sur l'algorithme sentencpiece [20].
Sentencepiece est un tokenizer de sous mots qui est indépendant du langage du corpus d'entraînement et qui tente de résoudre "open vocabulary problem". C'est-à-dire le traitement des mots rares qui ne font pas partie de son vocabulaire.

La figure 3.8 illustre le Pré-traitement du texte pour le modèle camemBERT.

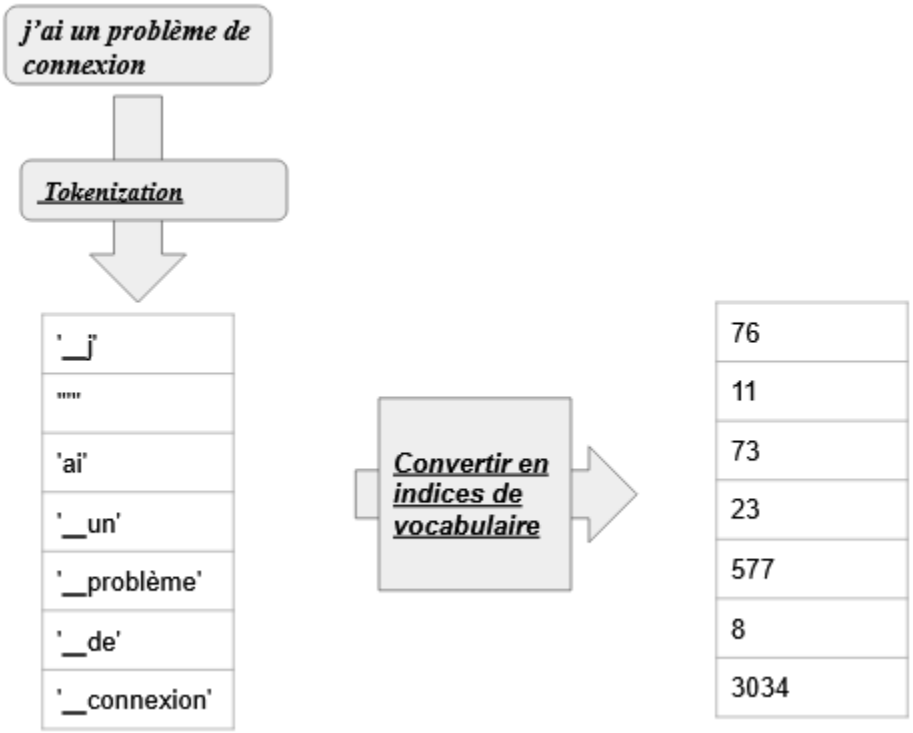


FIGURE 3.8 – Exemple de pré-traitement de texte.

3.2.5 Analyse exploratoire des données du modèle de classification d'intention

Pour la classe information générale, la majorité des données présentes dans la figure 3.16, étaient générés en utilisant "Chatito ".
La répartition des données pour chaque classe est présentée dans les figures suivantes.

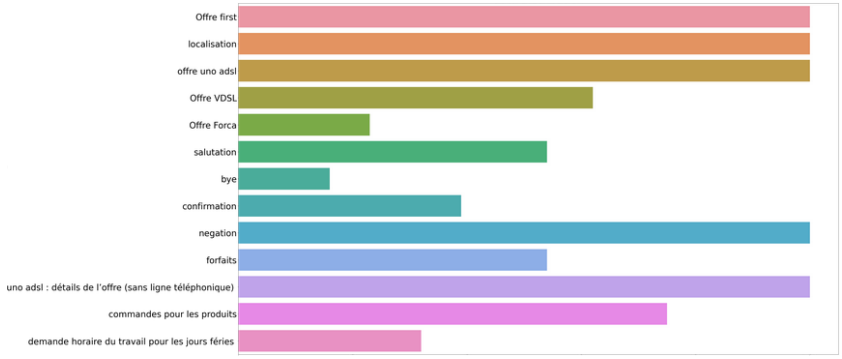


FIGURE 3.9 – Distribution des échantillons par intention dans l'ensemble de données de la classe information générale

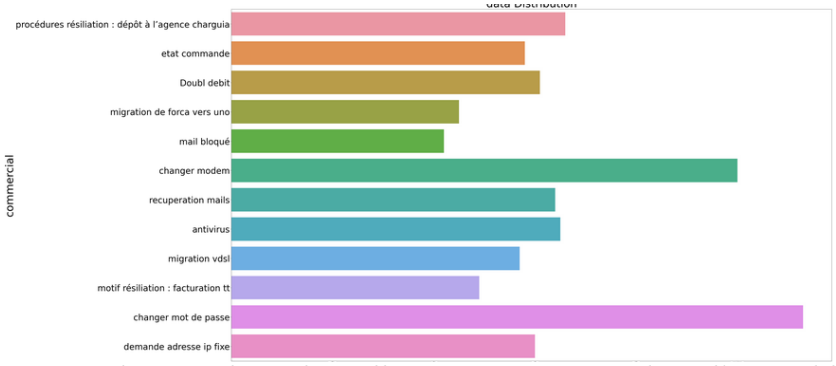


FIGURE 3.10 – Distribution des échantillons par intention dans l'ensemble de données de la classe commerciale

Les deux figures 3.20 et 3.21 mettent en évidence qui les données sont très biaisées avant les processus d'augmentation de données.

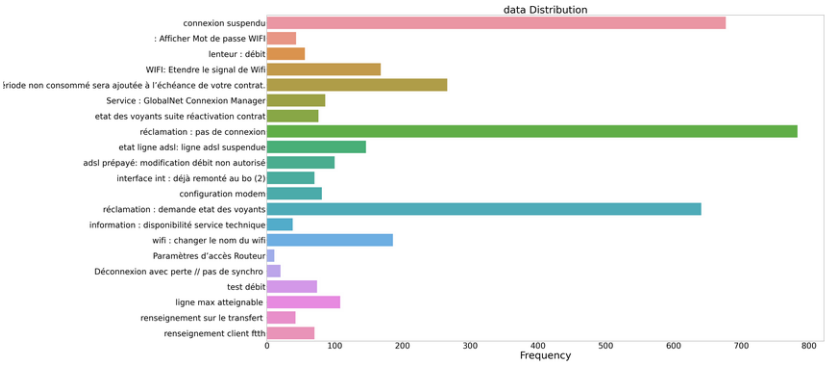


FIGURE 3.11 – Distribution des échantillons par intention dans l'ensemble de données de la classe technique

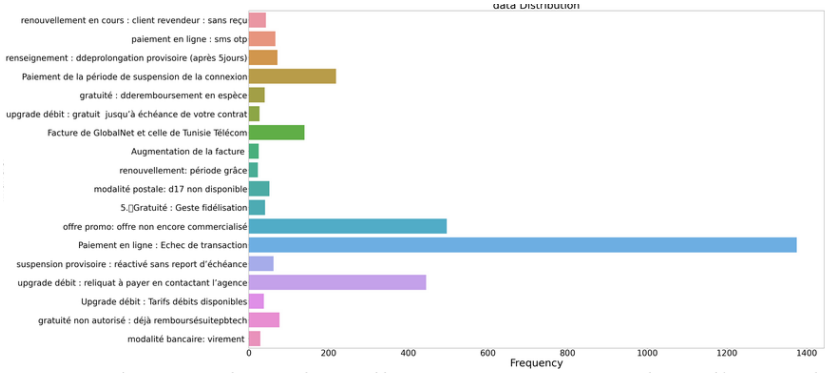


FIGURE 3.12 – Distribution des échantillons par intention dans l'ensemble de données de la classe financière

La figure 3.13 montre le nombre de tokens par question :

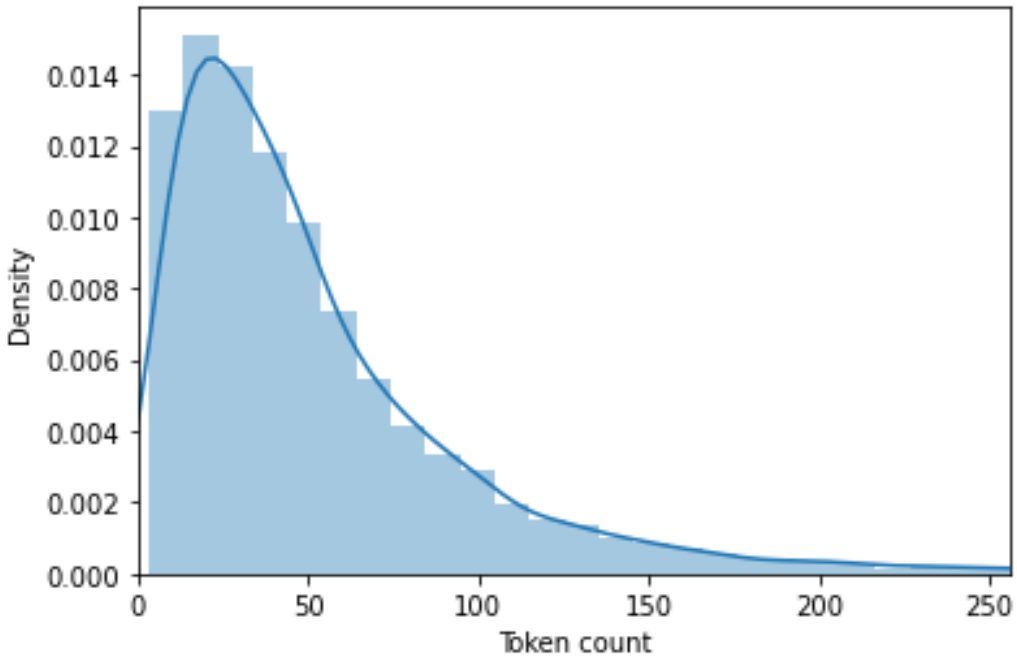


FIGURE 3.13 – Nombre de token par question

La plupart des questions contiennent moins de 250 tokens.
Cette information est très utile lors de l'utilisation des classifieurs basés sur transformers, car l'une des contraintes lors de la mise en œuvre de BERT ou CamemBERT est la longueur maximale de la séquence limitée à 512 tokens.



FIGURE 3.14 – Classe demande de payement

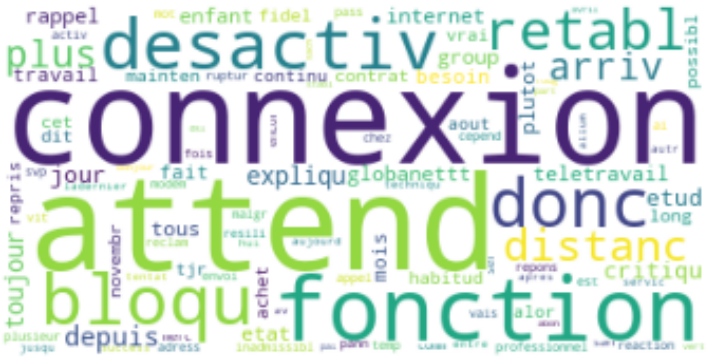


FIGURE 3.15 – Classe pas de connexion

Analyser les mots qui apparaissent le plus souvent dans chaque classe, est une étape importante en NLP qui nous permet dès le début d'avoir une indication sur la performance de modèle à implémenter. Les figures 3.14 et 3.15 illustrent la collection de mot qui existent pour les intentions "demande de payement" et "pas de connexion".

3.2.6 Formulation du problème : Classification de textes multi-classes

Considérons un ensemble de données D contenant des séquences d'échantillons de texte :

$$D = D_1, D_2, D_3, ...D_N$$

On considéra également un ensemble des classes.

$$Y = Y_1, Y_2, Y_3....Y_M$$

Où Di est le i^{th} échantillon de notre ensemble de données, qui est un segment de texte. Y est un ensemble des classes uniques et M est le nombre de classes dans l'ensemble de données.
La classification est la tâche d'approximation d'une fonction de correspondance (f) entre des variables d'entrée (D_i) et des variables de sortie discrètes (Y_i) :

$$f(D_i) = Y_i \tag{3.2}$$

Chaque séquence de texte a une classe unique à laquelle il appartient.

3.2.7 Modélisation

Cette section illustre les techniques utilisées afin de créer des modèles NLP authentiques et fiables.

Trois méthodes de classification ont été incluses dans la comparaison :

D'abord on a commencé par les modèles de machine learning avec la méthode TF-idf.
Ensuite, on va utiliser les modèles de deep Learning avec la technique de word embedding WORD2VEC (c'est la technique de représentation des caractéristiques).
Enfin, on va tester les modèles neuronaux reposant sur l'architecture du "Transformers".

3.2.7.1 Application de divers modèles de machine learning

La première approche consiste à utiliser différentes techniques d'apprentissage automatique qui ont été testées à l'aide de la bibliothèque Scikit-learn :
Pour ces modèles statiques, on utilise pour le prétraitement de chaque classifieur, une vectorisation basée sur la méthode TF-IDF.

Les différents modèles de classification utilisés sont les suivants :

1. Random Forest
2. Logistique régression
3. Naïve Bayes
4. SVM

3.2.7.2 Application de divers modèles de deep learning

Concernant le word embedding pour cette approche, on a la possibilité de choisir entre le modèle Continuous bag of words et Skip-Gram (présenté dans le chapitre 2).
on a choisi la première solution car on voulait une seule représentation de chaque mot par rapport au contexte global.
On a utilisé une taille de fenêtre de 3, cette valeur est utilisée pour définir les voisins du mot utilisé pour la capture du contexte.

LSTM avec la méthode CBOW :

L'architecture finale du modèle LSTM : construite à partir d'une couche d'entrée de type embedding de dimension 200 qui permet de caractériser chaque mot dans une séquence textuelle, elle est suivie par une couche SpatialDroupout1D qui permet d'effectuer un

dropout variationnel, puis une couche LSTM de 100 unités de mémoires, ajoutant encore une couche "fully connected" avec une fonction d'activation "Relu" et une couche Dropout pour éviter le problème d'overfitting. Enfin une couche de sortie avec une fonction d'activation "softmax".
La figure 3.16 montre l'architecture du LSTM.

```
Model: "sequential_2"
_________________________________________________________________
Layer (type)                 Output Shape              Param #
=================================================================
embedding_2 (Embedding)      (None, 200, 200)          31112400
_________________________________________________________________
spatial_dropout1d_1 (Spatial (None, 200, 200)          0
_________________________________________________________________
module_wrapper_1 (ModuleWrap (None, 100)               120400
_________________________________________________________________
dense_5 (Dense)              (None, 64)                6464
_________________________________________________________________
dropout_2 (Dropout)          (None, 64)                0
_________________________________________________________________
dense_6 (Dense)              (None, 4)                 260
=================================================================
Total params: 31,239,524
Trainable params: 127,124
Non-trainable params: 31,112,400
_________________________________________________________________
```

FIGURE 3.16 – Classifications LSTM à 4 contextes.

Bidirectional-LSTM et la méthode CBOW :

La deuxième architecture de deep Learning utilisé, basé sur Bidirectional-LSTM qui sont une extension des LSTM traditionnels.
Cette technique peut améliorer de manière significative les performances du modèle. En effet, deux LSTM sont utilisés au lieu d'un modèle LSTM traditionnel, cette technique peut fournir un contexte supplémentaire et permettre au modèle un apprentissage plus rapide et plus complet du problème.
La figure 3.17 illustre l'architecture finale du modèle Bidirectional-LSTM.

```
Model: "sequential_1"
_________________________________________________________________
Layer (type)                 Output Shape              Param #
=================================================================
embedding_1 (Embedding)      (None, 200, 200)          31112400
_________________________________________________________________
bidirectional_1 (Bidirection (None, 200)               240800
_________________________________________________________________
dropout_2 (Dropout)          (None, 200)               0
_________________________________________________________________
dense_2 (Dense)              (None, 64)                12864
_________________________________________________________________
dropout_3 (Dropout)          (None, 64)                0
_________________________________________________________________
dense_3 (Dense)              (None, 4)                 260
=================================================================
Total params: 31,366,324
Trainable params: 253,924
Non-trainable params: 31,112,400
_________________________________________________________________
```

FIGURE 3.17 – Architecture du modèle Bidirectional-LSTM

3.2.8 Application de divers modèles de transfert learning (BERT et CamemBERT)

Les deux modèles BERT et CamemBERT, ayant une architecture de type transformers, ont été implémentés en utilisant l'api pytorch et la bibliothéque de Hugging Face.

BERT :

Il existe deux variantes de modèle de langue pré-entraîné mBERT qui sont, mBERT-LARGE et mBERTBASE. Ces modèles diffèrent par leur architecture, comme le montre le tableau 3.1. Le Choix derrière l'utilisation du modèle "BERT-Base Multilingual Uncased." c'est la capacité limitée du matériel.

TABLE 3.1 – Architecture des différents modèles de mBERT

	mBERTBASE	mBERTLARGE
number of weights (word embeddings)	81315072	91812096
word embeddings dimension	[105879, 768]	[119547, 768]
Paramètres	168M	178M

Dans le but d'améliorer les résultats, on applique mBERT. En fait, l'innovation de mBERT est l'application de l'entraînement bidirectionnel de transformes. C'est un modèle basé sur le mécanisme d'attention et la modélisation du langage, en opposition aux traitement précédents qui traitaient une séquence de texte soit de gauche à droite, soit en combinant l'entraînement de gauche à droite puis de droite à gauche.

Les "transformers" nous fournissent un modèle BERTForSequenceClassification, il s'agit du modèle BERT avec une seule couche prédictive (single layer feed forward network) ajoutée au-dessus de modèle classique pour la classification avec une fonction softmax. Au fur et à mesure que nous "feed" les données d'entrée, le modèle mBERT pré-entraîné et la couche de classification supplémentaire non entraînée sont entraînés à notre tâche spécifique (classification).
Les informations acquises via le modèle mBERT ont été introduites dans l'apprentissage par transfert en utilisant la classe BERTForSequenceClassification de la bibliothèque Hugging Face[2].

CamemBERT :

Le modèle CamemBERT se base sur l'architecture RoBERTa[21], une variante de BERT. CamemBERT a été pré-formée dans un jeu de données français. On a utilisé CamemBERTforSequenceClassification qui est un transformers du modèle CamemBERT qui a de plus une couche prédictive pertinente au-dessus de la sortie du modèle, avec une fonction d'activation softmax.

3.3 Analyse sentimentale

On va enchaîner dans cette section avec les différentes étapes jusqu'à atteindre un modèle qui assure l'analyse des sentiments.

3.3.1 Importance de l'analyse sentimentale pour un chatbot

L'emploi et l'utilisation intensive des chatbots dans tous les secteurs d'activité, ont amené à devenir plus avancés et sophistiqués et à développer de nouvelles fonctions d'intelligence artificielle qui améliorent l'expérience client.
L'une de ces fonctionnalités est l'analyse des sentiments. Également connue sous le nom de "conversation mining". Cette technique permet aux chatbots de déterminer l'émotion du message d'un client et aide les entreprises à déterminer si les conversations avec les clients se déroulent bien ou non avec leurs chatbots. Donc cette valeur peut être utilisée aussi pour traduire le sentiment des clients à l'égard de la marque.

En outre, il a été estimé que les données non structurées représentent 80% du total mondial. De nouvelles données sont générées chaque jour sous la forme de chats, de conversations sur les médias sociaux, d'e-mails, de support client, d'articles, d'enquêtes et de documents. Cependant, il est très difficile d'analyser et de trier ces données. C'est là qu'intervient l'analyse des sentiments, qui aide les organisations à donner un sens à ces données non organisées en les étiquetant.
Cette section s'articule autour notre module d'analyse sentimentale implémenté pour le chatbot de Globalnet. Donc on passe en revue les étapes de mise en œuvre de ce système.

Le développement de la solution qu'on propose se compose de trois phases principales, comme le montre la figure 3.18



FIGURE 3.18 – Étapes de conception et de mise en œuvre d'un modèle d'analyse sentimentale.

3.3.2 Description des Données

L'ensemble de données utilisé dans cette partie représente d'environ de 17 000 commentaires qui sont collectés à travers une enquête de satisfaction faite par Globalnet dans le but de déterminer la meilleure façon d'améliorer ou de modifier ses produits et services. Le pipeline de nettoyage de ses données était similaire à la figure 3.1.

	ààààà	Unnamed: 1	Unnamed: 2	Unnamed: 3
0	Problème lié à Tunisie Télécom... comme d'habi...	NaN	NaN	NaN
1	toujours il y a des copures d'internet	NaN	NaN	NaN
2	problème de réception des mails.	NaN	NaN	NaN
3	NaN	NaN	NaN	NaN
4	le probléme n'est pas résolut!!!???	NaN	NaN	NaN
...	...	...	...	...
11119	Merci de traiter ma réclamation au plus vite p...	NaN	NaN	NaN

FIGURE 3.19 – Des commentaires d'une enquête de satisfaction

3.3.3 Analyse exploratoire des données

On présente et analyse ici quelques chiffres de la distribution des données. Comme le montre la figure 3.20, on a remarqué que la distribution des données est un peu déséquilibrée, 55.9% des commentaires insatisfaits et 44.1% qui sont satisfaits.

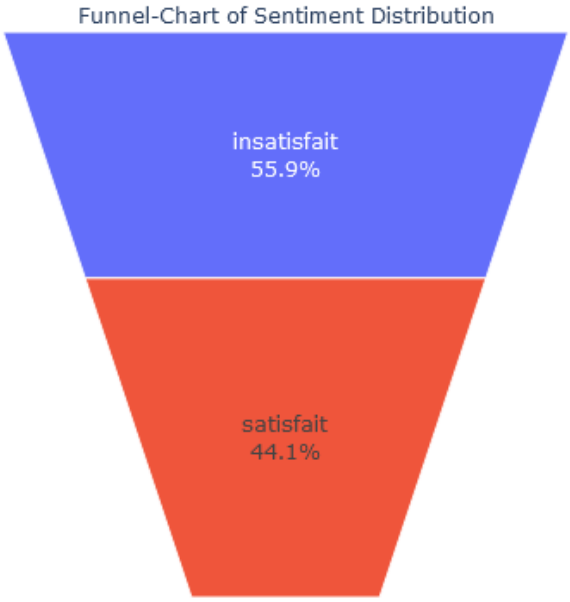


FIGURE 3.20 – Répartitions des données.

La figure 3.21 illustre la distribution de longueur de texte pour chaque catégorie. Le nombre de tokens maximale est environ 100.

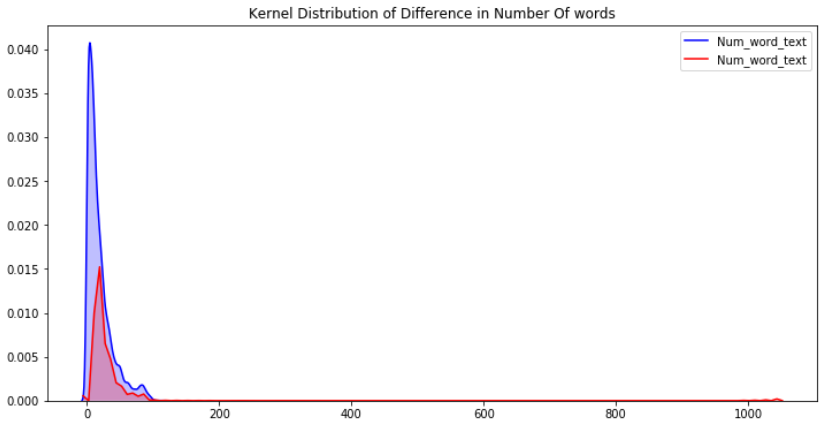


FIGURE 3.21 – Nombre de tokens par chaque classe

3.3.4 Modélisation

Pour cette section, on teste le modèle de machine learning Naïve Bayes, et les deux classifieurs basés sur des modèles neuronaux contextuels dynamiques BERT et CamemBERT pour notre système d'analyse des sentiments.

En ce qui concerne, les deux modèles BERT et CamemBERT (pré-entraîné) dans cette partie est un peu différente par rapport aux modèles de classification d'intention. En fait, pour ces derniers on a utilisé la bibliothèque des transformers (Hugging Face) qui fournit les classes BertForSequenceClassification et CamemBertForSequenceClassification qui sont conçue pour la tâche de classification.
Cependant, pour le modèle d'analyse sentimentale on a créé une nouvelle classe afin que on puisse spécifier notre propre choix de classification en utilisation pytorch native. On a créé ainsi deux classes BertClassifier et CamemBERTClassifier avec respectivement, le modèle BERT et le modèle CamemBERT qui fournit la représentation cachée des jetons d'entrée, puis on a ajouté une seule couche entièrement connectée (feed-forward neural network) avec une fonction softmax qui prend en entrée la représentation et prédit le sentiment. Les caractéristiques de notre classifieur sont les suivantes $D_{in}, H, D_{out} = 768, 50, 2$ représentent respectivement "hidden size" de BERT/CamemBERT , "Hidden size" de notre classifieur et les nombre des classes.
La sortie de la couche de transformers finale BERT est utilisée comme caractéristique de la séquence pour "feed" notre classifieur.

3.4 Environnement de travail

Pour mettre en œuvre les modèles qui on a conçus, on a utilisé un large éventail de technologies.

•Software :

Python :

Python est un langage de programmation de haut niveau conçu pour être facile à lire et simple à mettre en œuvre. Il est open source. Python peut fonctionner sur les systèmes Mac, Windows et Unix et a également été porté par les machines virtuelles Java et .NET. Il est livré avec plusieurs bibliothèques d'apprentissage automatique très attrayantes ainsi qu'un ensemble puissant de paquets pour les besoins analytiques. Cela nous a encouragé à choisir Python plutôt que d'autres langages de programmation. En outre, Python dispose d'une très grande communauté qui fournit un soutien à travers les forums.

Numpy :

Numpy est une bibliothèque Python, permet de traiter des tableaux et d'effectuer des opérations en algèbre linéaire, statistique et transformer de Fourier.

Pandas :

Pandas est un package Python open source largement utilisé pour les tâches de science des données, d'analyse de données et d'apprentissage automatique. Il est construit sur le package Numpy, qui prend en charge les tableaux multidimensionnels.

Scikit-learn :

Scikit-learn est l'une des bibliothèques les plus connues qui fournit une implémentation solide des algorithmes d'apprentissage automatique. Elle contient des outils simples et efficaces pour l'exploration et l'analyse des données.

Pytorch :

Pytorch est une bibliothèque open-source d'apprentissage automatique basée sur la bibliothèque Torch et crée par Facebook. Il est utilisée pour des applications telles que la vision par ordinateur et le traitement du langage naturel. Il s'agit d'un langage impératif, ce qui signifie les calculs sont exécutés immédiatement. Cette bibliothèque basée sur Python est construite pour offrir une flexibilité en tant que plateforme de développement d'apprentissage profond.

Transformers :

Transformers une bibliothèque open-source, fournit des milliers de modèles pré-entraînés qui permettent d'effectuer diverses tâches NLP. On peut citer la classification de texte, l'extraction d'informations, la reconnaissance d'entités de nom (NER), la traduction de texte, la génération de texte, la réponse à des questions et le sous-titrage d'images, etc.

Nlpaug :

NLpaug est une bibliothèque python open-source fournit des méthodes et techniques permettent d'augmenter les données textuelles pour les projets d'apprentissage automatique.

•Hardware :

Processeur : Intel (R) Core i7 @ 2.4 GHz.

RAM : 8 GO.

OS : Windows 10

Google colab :

C'est un outil puissant permettant aux utilisateurs de langage Python de créer et de partager des codes interactifs dynamique et exécutable, des visualisations des données, des textes et des équations. Il est proposé par Google (gratuit), basé sur Jupyter Notebook et destiné à la formation et à la recherche en apprentissage automatique. Cette plateforme permet l'entraînement de modèles d'apprentissage automatique directement dans le cloud. Il offre gratuitement un GPU et 12 Go de RAM.

3.5 Conclusion

Tout au long de ce chapitre, on a expliqué le processus de réalisation des différents modèles de classification (classification d'intention et analyse sentimentale). On a présenter dans une première partie le prétraitement ainsi que l'augmentation des données. Par la suite, on a présenté la conception des différents modèles. On a clôturé ce chapitre par une présentation de l'environnement de développement.
Dans le chapitre suivant, on va discuter les résultats obtenus par les différents modèles.

Chapitre 4

Résultats Expérimentaux des Modèles et Discussions

4.1 Introduction

Après avoir présenté dans les chapitres 1, 2 et 3 le cadre général du projet, l'état de l'art ainsi que les différents modèles utilisés, on exposera dans ce chapitre les différents résultats.
Dans la première section, on va présenter les métriques d'évaluation puis la deuxième sera consacrée à une comparaison des modèles et finalement, la troisième section sera dédiée pour l'analyse graphique.

4.2 Métriques d'évaluation

Les systèmes NLP sont généralement évalués en fonction de leurs performances sur l'ensemble de données de test pour une tâche spécifique. Pour le problème de classification, "l'accuracy" est la mesure d'évaluation commune, qui est définie comme suit ;

$$accuracy = \frac{TP + TN}{TP + TN + FP + FN} \tag{4.1}$$

où TP, TN, FP et FN sont respectivement le nombre de vrais positifs, de vrais négatifs, de faux positifs et de faux négatifs. En général, le nombre de prédictions vraies est divisé par le nombre de toutes les prédictions. Mais, il est nécessaire de disposer une autre métrique plus détaillée pour observer la performance par classe : F1-score.
C'est la moyenne harmonique de la précision et le rappel.
Il fournit donc une bonne estimation de la qualité globale d'un modèle.

Le F1-score est définie comme suit :

$$F1 - score = 2 * \frac{Precision * Rappel}{precision + Rappel} \tag{4.2}$$

avec Précision et Rappel sont définies comme suit :

$$Precision = P = \frac{TP}{TP + FP} \tag{4.3}$$

La précision mesure la fraction d'exemples prédits corrects de la catégorie divisée par le total des cas prédits dans cette catégorie. Contrairement à l'accuracy, la mesure de précision ne s'intéresse qu'à la proportion de bonnes classifications divisée par le nombre de cas prédits dans cette catégorie. Même si un ensemble de données est déséquilibré, il donnera la précision de la prédiction pour chaque catégorie.

$$Rappel = R = \frac{TP}{TP + FN} \tag{4.4}$$

Le rappel est le rapport entre les observations positives correctement prédites et toutes les observations de la catégorie.
•Matrice de confusion

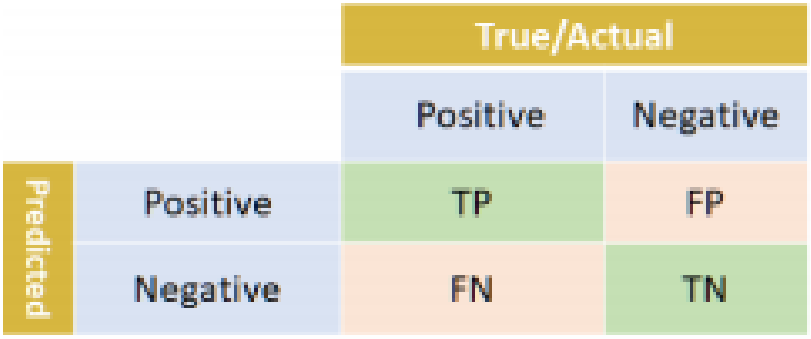


FIGURE 4.1 – Matrice de confusion Pour classification Binaire.

La matrice de confusion est une mesure de performance pour l'apprentissage automatique où la sortie peut être deux classes ou plus. Elle est extrêmement utile pour mesurer la précision, le rappel, accuracy, F1-score, la spécificité et d'autres métriques. Où,
Les vrais positifs (TP) correspondent aux échantillons correctement classés comme positifs.
Les faux positifs (FP) désignent les échantillons mal classés comme positifs.
Les vrais négatifs (TN) désignent les échantillons correctement classés comme négatifs.
Faux négatifs (FN) désignent les échantillons mal classés comme négatifs.

•La courbe ROC
La courbe ROC est crée en traçant le taux de vrais positifs (TPR) par rapport au taux de faux positifs (FPR), où TPR, FPR définie comme suit :

$$TPR = \frac{TP}{TP + FN} \tag{4.5}$$

$$FPR = \frac{FP}{FP + TN} \tag{4.6}$$

4.3 Résultats et discussions

Les résultats des différents classifieurs d'intention des utilisateurs sont reportés et analysés dans cette section.

4.3.1 Classification contextuelle et classification d'intention

Hyperparamètre

Un hyperparamètre est un paramètre dont la valeur est utilisée pour contrôler le processus d'apprentissage. Il a un impact important sur la performance du modèle.
Le réglage des paramètres a été effectué manuellement pour obtenir le meilleur résultat en termes d'accuracy.

4.3.1.1 Approche machine learning

Pour pouvoir travailler avec les modèles de machine learning on a utilisé la bibliothéque Scikit-learn.

Dans cette section, on va présenter les meilleurs hyper-paramètres obtenus, pour chaque modèle.

Pour Random Forest :

$n_{estimators} = 100$: représente le nombre d'arbres dans la forêt.
$max_{depth} = 5$: représente le profodeur maximale.

Pour SVM :

$C = 0.1$: un terme de régularisation.
$kernel = rbf$: (Radial Basis Function en anglais), c'est l'un des noyaux non linéaires le plus couramment utilisé lorsque la fonction de décision est non-linéaire. C'est un noyau Gaussien qui permet de mesurer la similarité entre deux exemples.
L'idée derrière notion de noyau est d'appliquer une transformation sur les données pour pouvoir les séparer.
$Gamma = 0.5$: permet de contrôler la distance de l'influence d'un seul point d'entraînement.

Pour les deux modèles (régression logistique et naïve bayes), on a utilisé l'algorithme multinomial.

TABLE 4.1 – Comparaison des performances des classifieurs statistiques

Modèle	TF-IDF + SVM	TF-IDF + RL	TF-IDF+Naive Bayes	TF-IDF+RF
contextuel	0.78	0.73	0.60	0.71
financier	0.84	0.81	0.75	0.80
commercial	0.80	0.79	0.77	0.59
technique	0.76	0.72	0,74	0.65
information générale	0.88	0.88	0.86	0.62

Le tableau 4.1 confirme que TF-IDF représente une méthode importante pour l'extraction de caractéristiques (Feature extraction).
On a combiné cette méthode avec un ensemble diversifié d'algorithmes d'apprentissage automatique connus par leurs performances dans la tâche de classification de texte.
Il est clair d'après le tableau 4.1 que SVM sur-performe les simples classifieurs comme Random forest, régression logistique et naïve bayes.

4.3.1.2 Approche deep Learning

Pour l'approche "Deep Learning", comme il s'agit d'un problème de classification multi-classes $categorical-crossentropy$ est utilisé comme une fonction de perte, et l'optimiseur Adam a été choisi pour l'implémentation des modèles.
Adam [18] : est un algorithme d'optimisation du taux d'apprentissage adaptatif qui a été conçu spécifiquement pour la formation des réseaux de neurones profonds.

Fine Tuning LSTM :

Pour les paramètres, on a choisi ceux qui nous donnent une perte totale minimale.
$Dropout = 0.3$
$Learning - rate = 0.0001$
$Batch - size = 64$
$epochs = 5$

TABLE 4.2 – Résultats de modèle traditionnel de deep Learning "LSTM"

Modèle	Accuracy	Loss	F1-score
contextuel	0.71	0.2	0.71
financier	0.65	0.8	0.54
commercial	0.61	1.3	0.55
technique	0.57	0.9	0.40
information générale	0.67	0.63	0.71

Fine Tuning BILSTM :

Après avoir implémenté le modèle BILSTM, on a commencé à expérimenter le réglage des hyperparamètres. On a implementé un BILSTM avec :

$learning - rate = 0,0003$
$batch - size = 64$
$epochs = 5$
$Dropout = 0.25$

TABLE 4.3 – Résultats de modèle traditionnel de deep Learning "BILSTM"

Modèle	Accuracy	Loss	F1-score
contextuel	0.74	0.2	0.76
Financier	0.77	0.5	0.64
Commercial	0.68	0.8	0.69
Technique	0.68	0.8	0.58
Information générale	0.7	0.3	0.67

Les classifieurs basés sur les réseaux de neurones classiques, LSTM et BILSTM donnent des résultats inférieurs par rapport aux classifieurs statistiques (SVM).
Le problème revient peut-être au fait que les réseaux de neurones traditionnels dépendent d'une grande quantité des données étiquetées. Une chose qui n'est pas fournie dans notre cas.

4.3.1.3 Approche transfert learning

Fine Tuning mBERT :

En ce qui concerne les hyperparamètres, l'article original de BERT [13] recommande :

"$Batch - size$" : $[16, 32]$
"$Learning - rate$" : $[2e-5, 3e-5, 5e-5]$
"$Nombre - de - epochs$" : $[2, 3, 4]$

Cependant, les meilleurs résultats ont été résumés dans le tableau 4.4.

TABLE 4.4 – Meilleurs hyperparamètres de modèle mBERT

Taille de données de la formation	0.75
époques	4
longueur maximale	250
Taille du lot	16
learning rate	5e-5
eps	1e-8
Optimiseur	AdamW

On a choisi l'optimiseur AdamW [17] (Adam with weight decay fix) avec des valeurs par défaut pour betas et epsilon = 1e-8.

TABLE 4.5 – Résultats du modèle de langage mBERT

Modèle	Accuracy	Loss	F1-score
contextuel	0.93	0.26	0.93
financier	0.52	0.75	0.46
commercial	0.480	0.8	0.50
technique	0.58	0.8	0.47
information générale	0.798	0.4	0.73

En utilisant mBERT le résultat était moyen d'après le tableau 4.5. Ceci se justifie par le fait que généralement les modèles multi-langues sont performants pour l'anglais.[24]

Fine Tuning CamemBERT

En effectuant quelques études préliminaires, On a identifié les paramètres les plus importants, présentés dans le tableau 4.6

TABLE 4.6 – Meilleurs hyperparamètres de modèle CamemBERT

Taille de données de la formation	0.75
époques	4
longueur maximale	250
Taille du lot	16
learning rate	2e-5
eps	10e-8
Optimiseur	AdamW

TABLE 4.7 – Résultats du modèle de langage CamemBERT

Modèle	Accuracy	Loss	F1-score
contextuel	0.96	0.16	0.95
financier	0.82	0.13	0.78
commercial	0.80	0.19	0.79
technique	0.82	0.13	0.73
information générale	0.85	0.18	0.81

Le modèle camemBERT, utilisant la partie Encoder de l'architecture Transformers, a été entraîné sur un grand corpus des données en langue française. Ce premier a donné des résultats meilleurs que les autres types de classifieurs. L'écart entre ce modèle et les modèles précédents peut néanmoins être plus ou moins important.
Le tableau 4.8 montre les valeurs de rappel, de précision et de F1-score pour la tâche de classification contextuelle.

TABLE 4.8 – Rappel, Précision et F1-score du modèle CamemBERT

Classe	Précision	Rappel	f1-score
Financière	0.89	0.95	0.92
Commerciale	0.95	0.92	0.94
technique	0.99	0.94	0.96
Information générale	0.99	0.99	0.99

Le tableau 4.8 montre que le modèle CamemBERT effectue efficacement la tâche de classification pour les textes français et il surpasse ainsi les autres modèles évoqués précédemment. Ce dernier apporte une grande valeur ajoutée, puisqu'il est entraîné uniquement sur des corpus français plus gros que la partie française du corpus sur laquelle est pré-entraîné le BERT multilingue.

4.3.2 Analyse sentimentale

Comme on utilise des modèles de classification binaire, on choisit les métriques d'évaluation pour notre modèle : accuracy et l'AUC - ROC.

Fine Tuning différentes modèles d'analyse sentimentale

Les données sont divisées en 80% pour l'apprentissage et 20% pour la validation, afin de mieux évaluer les performances de nos modèles.

Les principaux paramètres utilisés pour le Fine-Tuning sont représentés ici :

Fine-Tuning BERT :

$$epochs : 4, BatchSize : 32, Optimiseur : AdamW, lr = 5e-5, eps = 1e-8$$

On "Unfreeze" deux couches du modèle.

Fine-Tuning CamemBERT :

$$epochs : 4, Batchsize : 16, Optimiseur : AdamW, lr = 10e-5, eps = 1e-8,$$

On "freeze" l'ensemble des couches du modèle.

Fine-Tuning Naive-Bayes+TF-IDF :

Pour $tf - idf$, on a utilisé $ngram - range = (1, 3)$ cela signifie que nos caractéristiques peuvent être constituées de un, de deux ou même de trois mots contigus afin de capturer le contexte de la phrase et avec $alpha = 1.8$.

4.3.2.1 Résultats et discussions : analyse sentimentale

Le tableau suivant illustre une comparaison des approches d'analyse sentimentale en se référant à des métriques qui sont l'accuracy, l'AUC-ROC et le temps d'exécution.

TABLE 4.9 – Résultats des tests du modèle d'analyse sentimentale.

Modèle	Accuracy	AUC-ROC	Temps d'exécution(s)
Naïve Bayes	0.73	0.81	22.7
BERT	0.80	0.90	158.81
CamemBERT	0.91	0.97	92.8

En ajoutant un simple classifieur de réseau neuronal (une seul couche fully connected) au-dessus de BERT puis en faisant le "Fine Tuning", on peut atteindre des performances proches de l'état de l'art, soit 0.07 mieux en terme d'accuracy que la méthode de Naive Bayes. En plus, pour le modèle CamemBERT (le modèle français de BERT) le résultat a été amélioré par rapport à Naïve Bayes (avec 0.18 de plus en terme d'accuracy).

En outre, bien que l'architecture de CamemBERT soit très grande et compliquée (en considérant les paramètres et les poids), le "Fine-Tuning" était nécessaire chaque 2 à 5 époques. Ce résultat se justifie par le fait que CamemBERT a été entraîné sur une quantité énorme de corpus français. Cette performance impressionnante obtenue en peu de temps, avec une petite quantité de données, confirme le fait que CamemBERT est l'un des modèles NLP les plus puissants disponibles à l'heure actuelle pour la langue française.[22]

La figure suivante présente l'architecture finale de notre système d'analyse sentimentale.

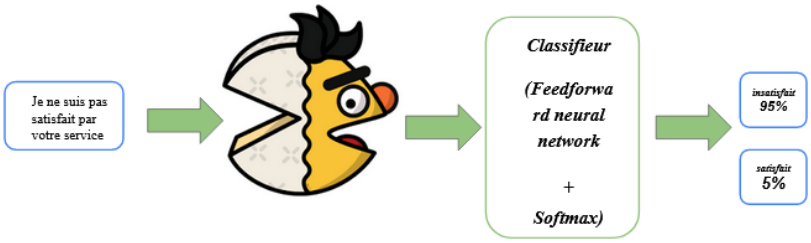


FIGURE 4.2 – Architecture de modèle d'analyse sentimentale CamemBERT.

4.4 Visualisation

Dans cette section, on va présenter quelques graphiques pour une meilleure visualisation des résultats.

4.4.1 Classification contextuelle et classification d'intention

La figure 4.3 présente la perte de formation (Training Loss) et la perte de validation (Validation Loss) au cours de formation du modèle de classification contextuelle.

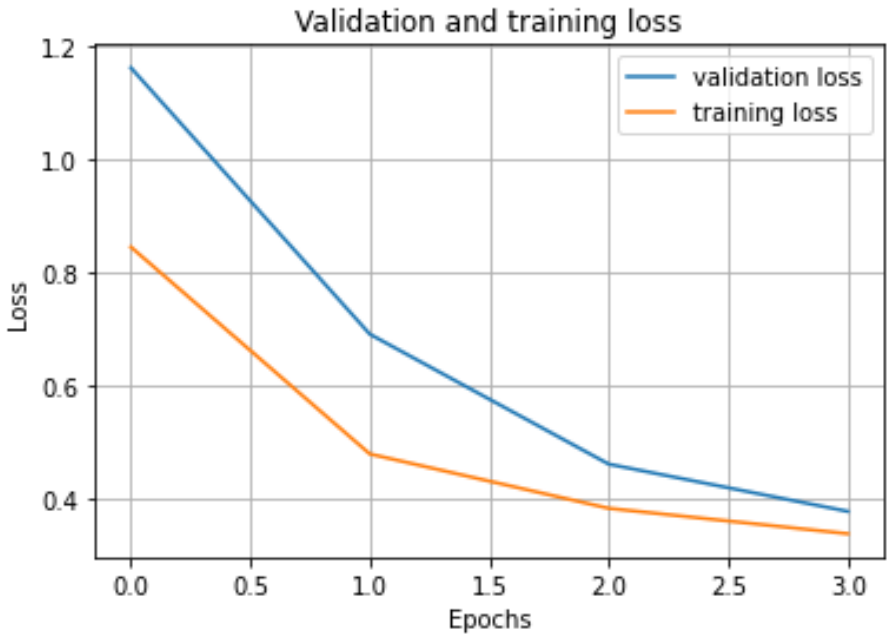


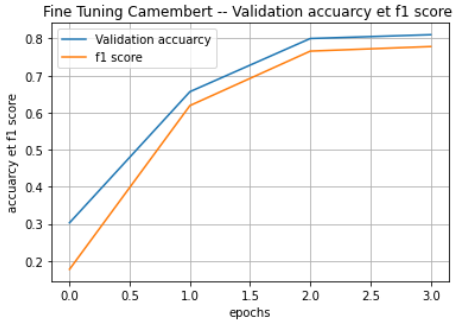
FIGURE 4.3 – Évolution de la perte de formation et de validation

Le tracet en bleu signale la perte globale du modèle à travers le temps sur le Training data. Celui en orange représente la même information mais sur un Test data.
cette figure montre que la perte d'apprentissage et de validation diminuent jusqu'à un point de stabilité avec un écart minimal entre les deux valeurs de perte finales. En plus,

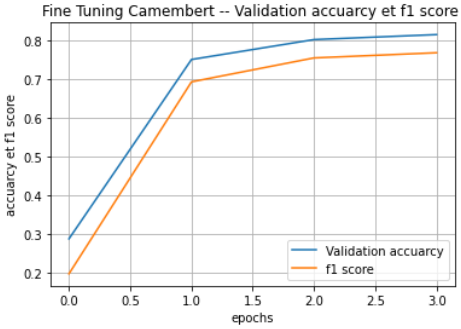
on remarque que la perte du modèle est toujours plus faible sur l'ensemble de données d'apprentissage que sur l'ensemble de données de validation. Ainsi, il existe un certain écart entre les courbes d'apprentissage de perte de train et de validation .
Cet écart est appelé « écart de généralisation ».

On peut conclure que notre modèle est au juste milieu, c'est-à-dire qu'il ne souffre ni d'Underfitting, ni d'Overfitting . Donc ,il s'agit bien d'un modèle généralisable qui a la capacité à s'adapter correctement à de nouvelles données qui n'étaient pas visibles au préalable, des données qui sont issues de la même distribution que celle utilisée pour créer ce modèle.

Les figures suivantes illustrent l'évolution de F1-score et accuracy au moment de formation des modèles.



(a) Modèle de classification d'intention commerciale



(b) Modèle de classification d'intention financière

FIGURE 4.4 – Relation entre (F1-score/accuracy) et époques.

Comme on peut le voir sur la figure 4.4, lorsque le nombre d'époques augmente, le F1-score et la valeur d'accuracy augmentent également, ce qui signifie que la performance de classification du modèle s'améliore. Lorsque la valeur d'époque est de 4, on obtient la meilleure performance du modèle.

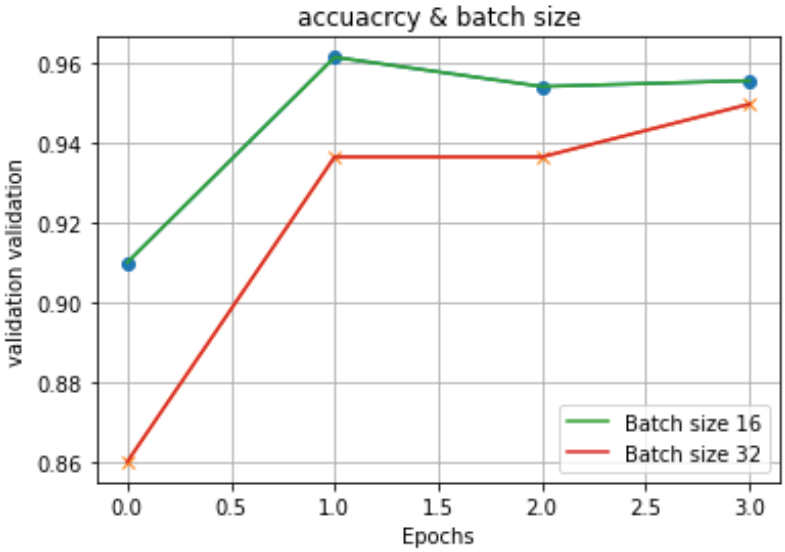


FIGURE 4.5 – Accuracy pour différentes tailles du lot

La figure 4.5 montre que la meilleure valeur d'accuracy est d'environ 96% obtenue avec une taille de lot égale à 16 et un nombre d'itérations égal à 4.
Pour mieux visualiser les performances de chaque classe, il est nécessaire de tracer la matrice de confusion obtenue dans la figure suivante :

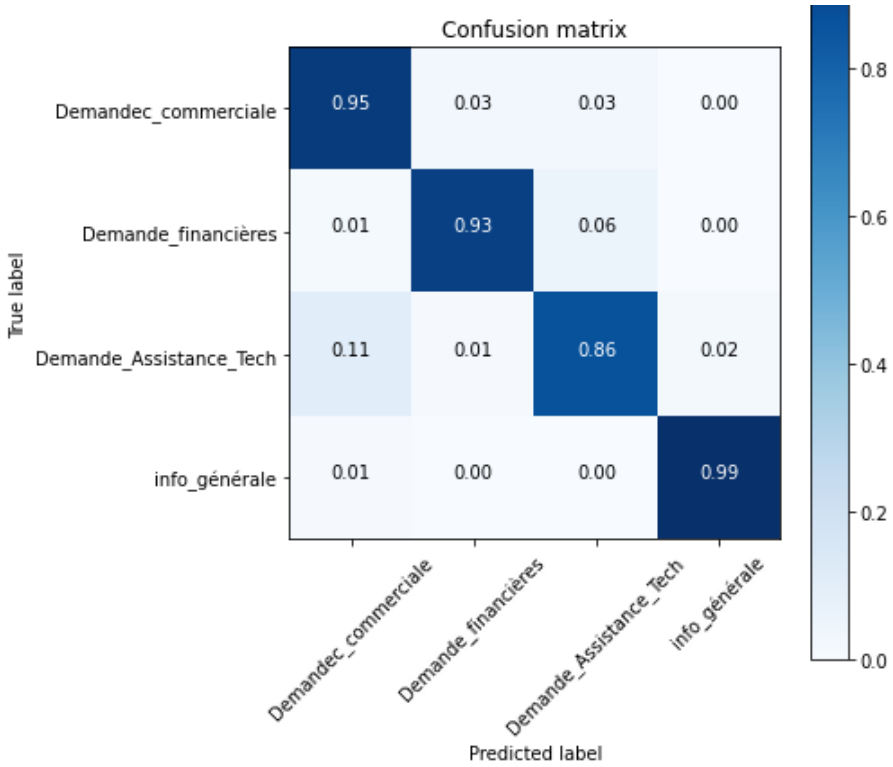


FIGURE 4.6 – Évaluation du modèle CamemBERT : Matrice de confusion

La matrice de confusion 4.6 montre que la précision du modèle de classification contextuelle est très acceptable et il peut bien distinguer entre les différentes classes. La précision est d'environ 99% pour les informations générales (la classe la plus distinguée). La classe des problèmes techniques donne environ 96%, suivie par la classe des problèmes commerciaux avec une précision 95%. En dernier lieu, on trouve la classe financière avec 93% précision. Une précision élevée explique la super performance de CamemBERT pour distinguer entre les différentes classes.

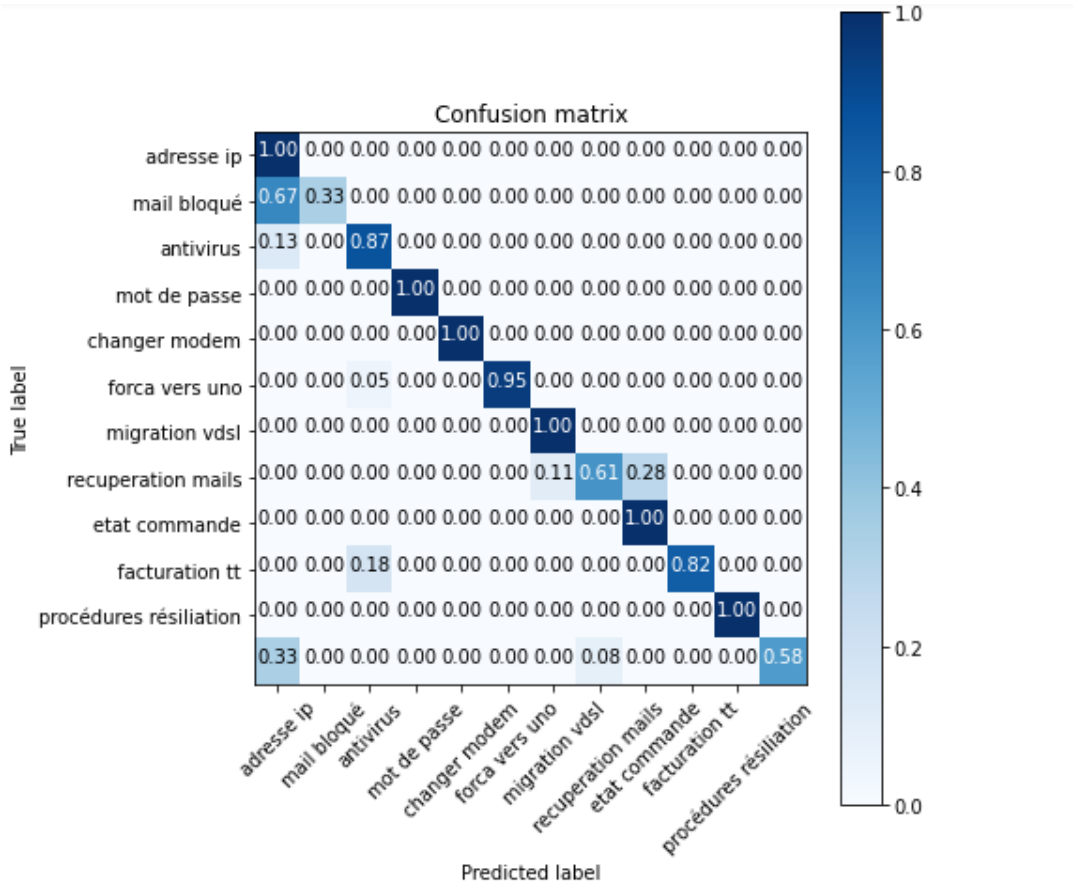


FIGURE 4.7 – Matrice de confusion normalisée montrant toutes les entités de l'ensemble de données de classe commerciale pour le modèle CamemBERT

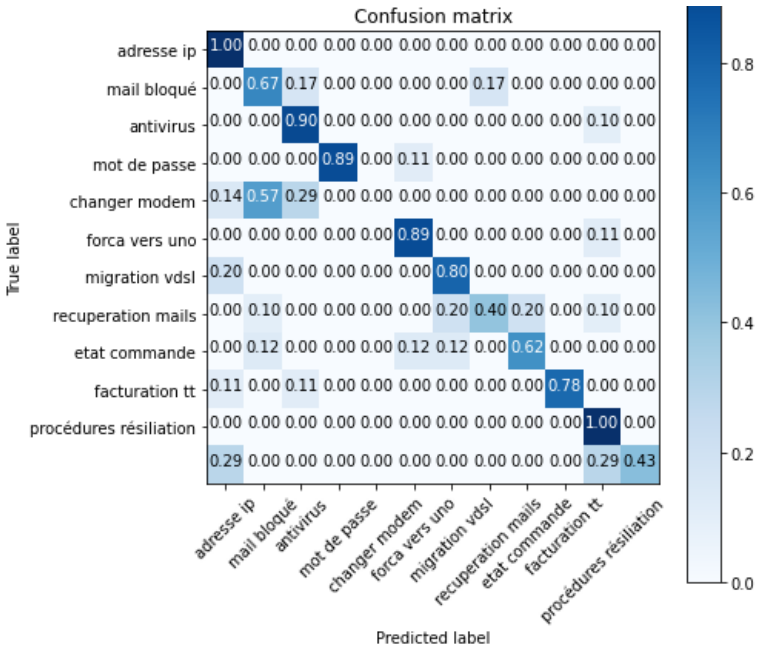


FIGURE 4.8 – Classe Commerciale utilisant mBert

Le modèle Camembert reste extrêmement important à discuter et à évaluer car il est basé sur les réseaux de neurones. CamemBERT donne des résultats très satisfaisants, en les comparant avec les résultats de l'état de l'art dans la tâche de classification.

On a mesuré les performances de CamemBERT par rapport au modèle mBERT (multilingue BERT) pour la tâche de classification en remarquant que CAMEMBERT a contribué à établir l'état de l'art et surpasse le modèle mBERT. La figure 4.8 met en évidence, que le Transformer spécialement pré-entraîné en français, obtient de meilleurs résultats que les modèles multilingues de BERT pour la classification d'intention du modèle attribué à la classe commerciale.
L'intelligence artificielle de CamemBERT maîtrise également la grammaire et la syntaxe françaises, et possède un vocabulaire riche.
Les chercheurs l'affirment aussi que le taux de réussite de cette intelligence artificielle basée sur le CamemBERT est de plus de 99% [22] (presque la même performance a été obtenue au cours de ce projet.)

La figure 4.9 montre comment chaque couche du modèle CamemBERT sépare les données, d'où grâce à cette figure on peut extraire les caractéristiques de chaque cluster ou la distribution des embeddings. En fait, la sortie de chacun de ces couches d'ENCODER peut être vue comme une séquence d'embedding contextualisé. Chaque séquence est ensuite introduite dans la couche suivante.

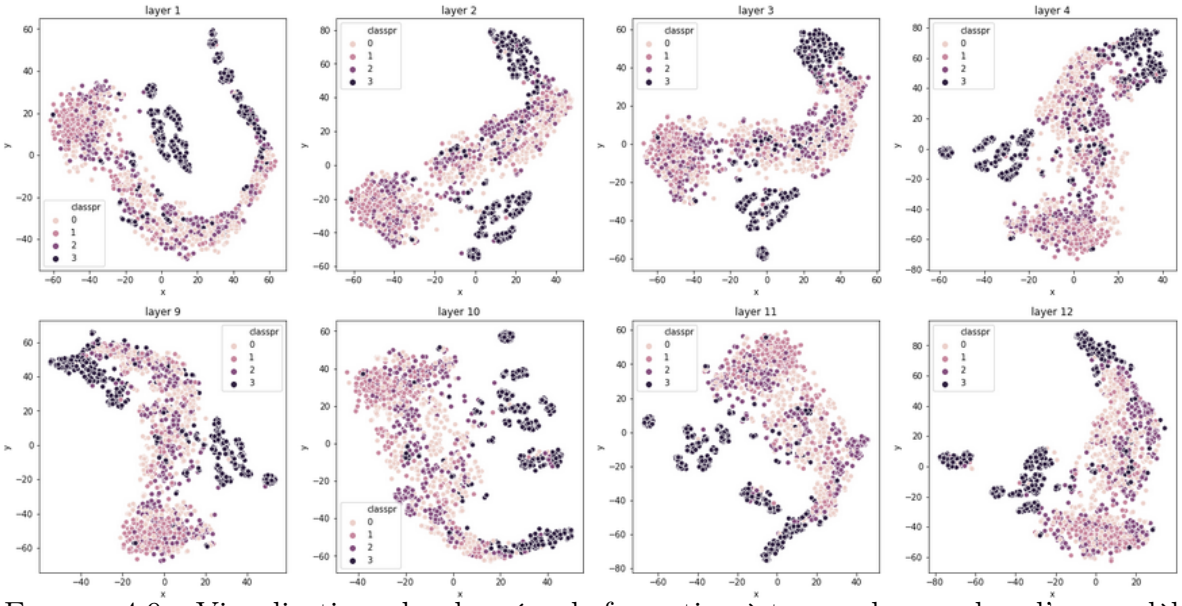


FIGURE 4.9 – Visualisations des données de formation à travers les couches d'un modèle CamemBERT

La distance entre les "Word embedding" reflète le degré de similitude entre ces mots.

4.4.2 Analyse sentimentale

Les deux figures suivantes présentent les courbes ROC de modèles CamemBERT et Naïve Bayes.

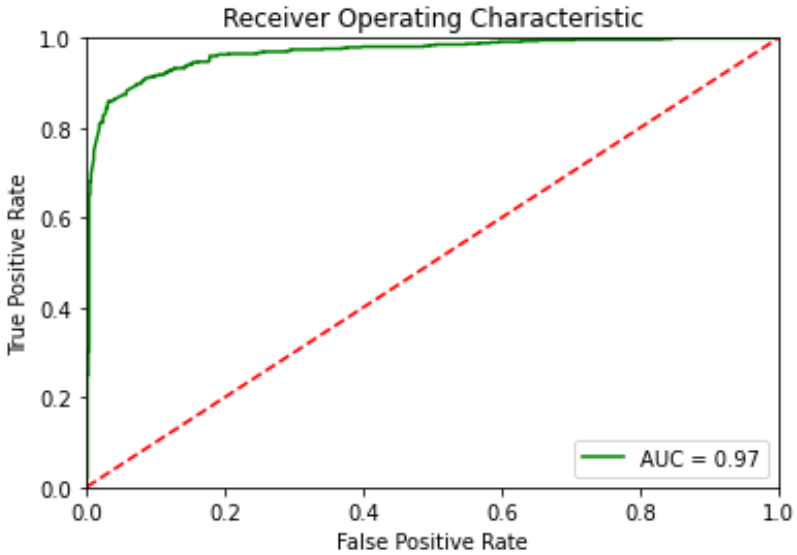


FIGURE 4.10 – CamemBERT AUC ROC

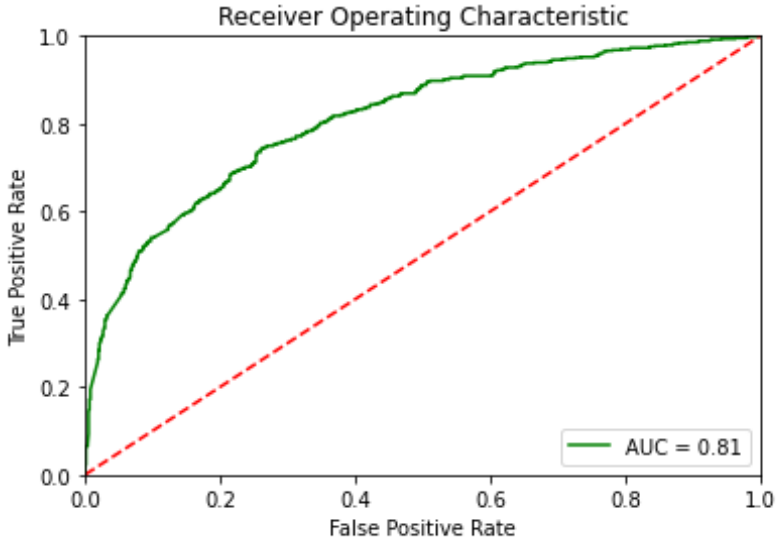


FIGURE 4.11 – Naïve Bayes AUC ROC

AUC ROC est une mesure de l'utilité d'un classifieur, où une aire plus grande signifie que le modèle est très performant pour la tâche de classification.
On trouve que CamemBERT $AUC - ROC = 97\%$ et $NaiveBayesAUC - ROC = 81\%$ C'est à dire que le classifieur CamemBERT est capable de distinguer entre les valeurs de classe positive et des valeurs de classe négative mieux que Naive Bayes. En effet, CamemBERT est capable de détecter un plus grand nombre de vrais positifs et de vrais négatifs que de faux négatifs et de faux positifs.

En analysant les résultats, il est très évident que les modèles de langage basés sur CamemBERT surpassent les modèles actuels de l'état de l'art, et ce même avec une base des données limitée disponible pour la formation. Cela n'est pas surprenant, compte tenu des résultats de l'article proposant Camembert[22]. Notre classifieur multi-classes a bien fonctionné même pour le classifieur binaire (analyse sentimentale).

4.5 Conclusion

Au cours de ce chapitre, on a cherché à comparer et à évaluer les différents modèles de classification de texte. On a commencé tout d'abord par les classifieurs statiques (SVM, RF, RL, Naïve Bayes) avec la méthode TF-IDF, puis les classifieurs basés sur des modèles neuronaux récurrents (LSTM et BILSTM) avec CBOW. Enfin, les classifieurs basés sur des modèles neuronaux contextuels dynamiques (Camembert, mBERT).
Cela prouve la capacité et l'efficacité de Transformers, plus précisément le modèle CamemBert pour la tâche de classification textuelle.
Le dernier chapitre, sera consacré à développer l'interface conversationnelle.

Chapitre 5

Développement de L'interface Conversationnelle

5.1 Introduction

Ce chapitre représente la dernière partie du projet qui consiste à développer une interface conversationnelle pour le déploiement de nos modèles de classification, d'intention et d'analyse sentimentale. La réalisation de cette partie a commencé par l'analyse des besoins fonctionnels et non fonctionnels suivie par une présentation des diagrammes UML. On clôturera par le développement de l'interface conversationnelle.

5.2 Analyse et spécification des besoins

La spécification des besoins est une étape fondamentale au début de chaque projet. Son objectif est d'essayer de développer une application adéquate basant sur une description générale des fonctionnalités du système.
Le système à réaliser doit satisfaire les exigences des utilisateurs.
On décrit dans ce qui suit tous les besoins fonctionnels et non fonctionnels à tous les acteurs du système.

5.2.1 Identification des acteurs

Un acteur est une personne, un matériel ou un logiciel qui interagit avec le système.
Dans notre projet les acteurs sont les suivants :

- L'utilisateur : les internautes qui peuvent être des clients ou veulent devenir chez Globalnet.

- Les employées chez Globalnet.

5.2.2 Spécifications des besoins

Besoins fonctionnels :

L'objectif de ce projet de fin d'études est la mise en place d'une solution d'Agent Conversationnel intelligent permettant de :

1. Répondre aux demandes des clients les plus fréquents quelques soit la demande de son identifiant (clé d'activation), activation d'antivirus, les problèmes de connexion, Les causes d'une connexion ADSL lente, les problèmes de connexion avec un modem..etc.

2. Interaction avec la base de données afin de répondre à certaines questions.

3. Sauvegarder les remarques et les réclamations dans une base de données afin d'améliorer l'expérience client.

4. Informer le client sur le mode de paiement, la politique de remboursement et les nouvelles promotions.

5. Mesurer la satisfaction du client.

6. Vérifier les contrats et les factures.

7. Notifier l'utilisateur à propos de l'état de sa demande par un e-mail.

8. Aider l'utilisateur à gérer son compte MyGnet.

Besoins non fonctionnels :

•La fiabilité : l'agent Virtuel doit satisfaire tous les besoins fonctionnels préalablement décrits et fonctionner correctement à tout moment.

•La rapidité : l'agent doit répondre dans un court laps de temps.

•Extensibilité : l'interface conversationnelle doit permettre à tout moment l'ajout d'autres modules.

•La sécurité : assurer la sécurité des données personnelles.

•La Clarté : l'interface conversationnelle doit être simple à utiliser.

Aussi l'agent virtuel doit :
Comprendre et traiter le langage naturel (français) pour fournir des réponses plus personnalisées.

5.2.3 Conception et modélisation UML

La modélisation est un élément essentiel pour les projets logiciels. Elle nous aide à mieux concevoir nos systèmes. Un logiciel est aussi bon que sa conception. Une conception médiocre conduira le plus souvent à un logiciel médiocre, tandis qu'une excellente conception facilitera la réalisation d'une excellente application.

5.2.3.1 Diagramme de cas d'utilisation

Ce type de diagramme est la première étape d'analyse UML, utilisé pour décrire les principales fonctions du système en représentant l'unité d'interaction discrète entre le client et le chatbot :

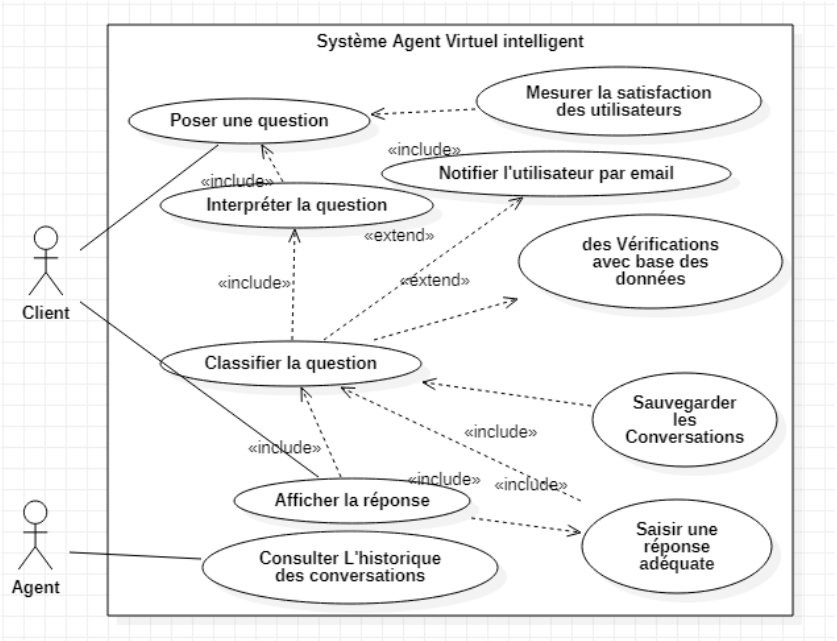


FIGURE 5.1 – Diagramme de cas d'utilisation global.

Tout d'abord, le client pose une question dans l'interface conversationnelle, puis l'agent virtuel interprète et fait la classification d'intention de l'utilisateur afin qu'il puisse choisir

la réponse appropriée. Des notifications par e-mail automatisées peuvent être envoyées et même des vérifications peuvent être se produire selon la demande du client.

5.2.3.2 Diagramme de communication

Les diagrammes de communication UML, précédemment appelés diagrammes de collaboration, sont un type de diagramme de comportement qui montre les interactions qui ont lieu entre les objets d'un logiciel ou d'un système. Ce type de diagramme met l'accent sur les messages échangés entre les objets.
La figure 5.2 montre le diagramme de communication qui présente les interactions entre les différents objets du système (client, chatbot et la base de données) par les différents liens.

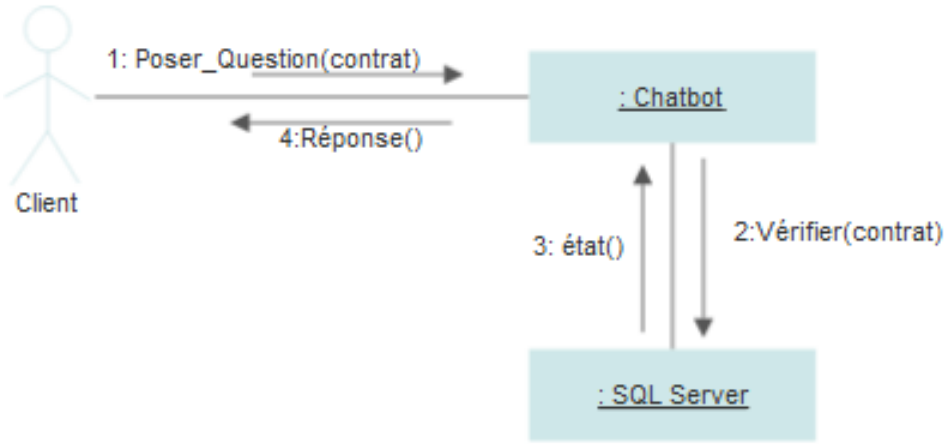


FIGURE 5.2 – Diagramme de communication.

L'internaute (le client) demande l'état du son contrat, ensuite le chatbot doit vérifier si le contrat est actif ou non dans la base de données, puis il envoie au client des informations nécessaires(état, date échéance...etc).

5.2.3.3 Diagramme de séquence

Les diagrammes de séquence sont utilisés pour décrire dans une séquence temporelle comment les classes impliquées fonctionnent entre elles et quels messages sont échangés pendant un scénario spécifique qui pourrait se produire dans le système analysé.

•Diagramme de séquence "la suspension de la connexion" :
La figure 5.3 illustre le comportement du système lors de la demande de "raison de la suspension de la connexion".

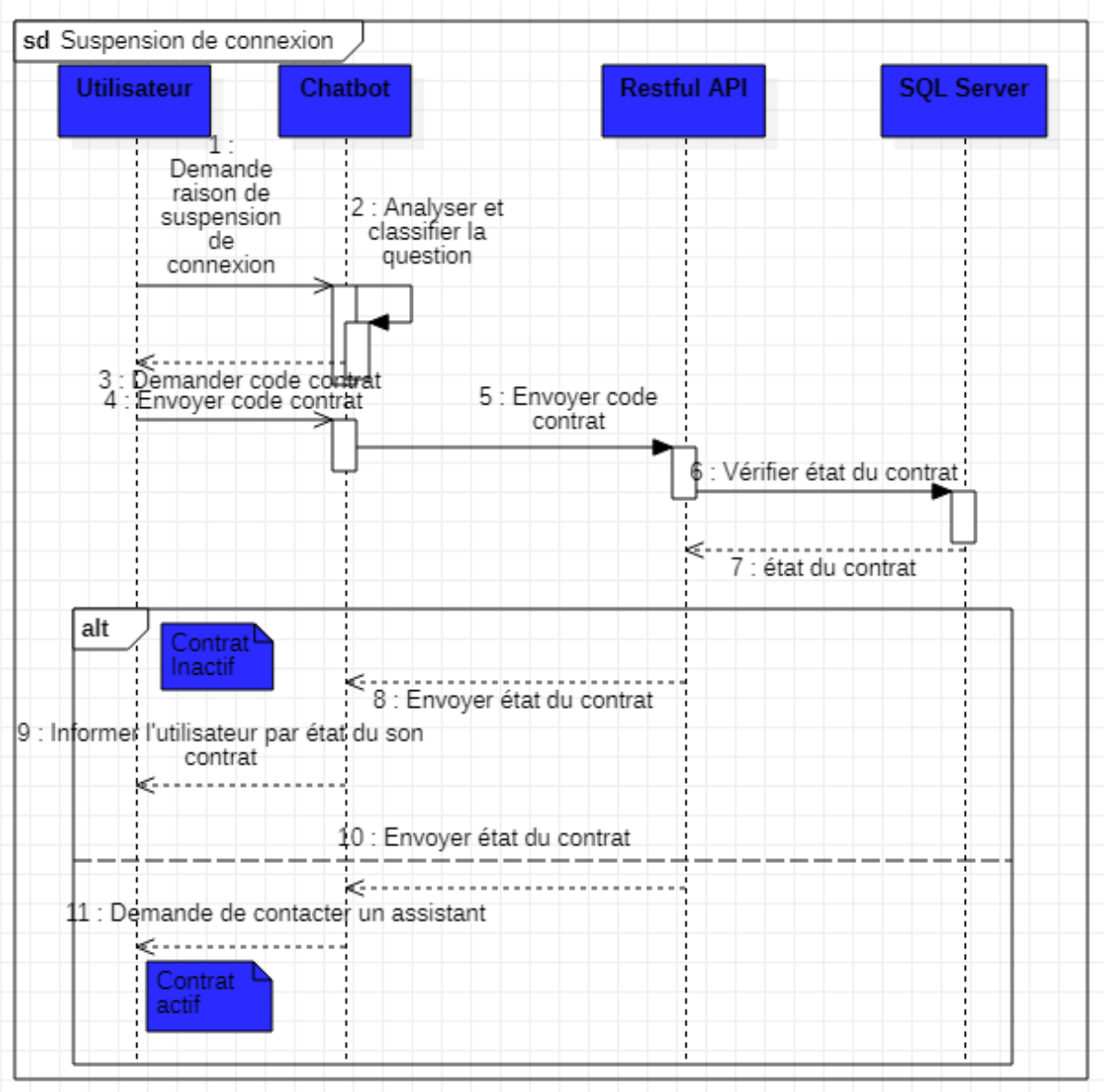


FIGURE 5.3 – Diagramme de séquence.

•**Description textuelle**

- **Acteurs** : tous client qui a un problème de suspension de la connexion
- **Pré-condition** :
  Demande raison de suspension de la connexion.

- **Poste-condition** :
  Utilisateur reçu la raison de ce problème.
- **Scénario nominal :**

  1. Le client tape une question dans une interface conversationnelle.
  2. Le chatbot fait le traitement nécessaire et demande le code contrat.
  3. Le client envoie le code contrat.
  4. API Restful utilise le code pour vérifier l'état du contrat dans la base de données.
  5. Si le contrat est actif : l'agent demande au client de contacter l'équipe technique.
  6. Si Le contrat est inactif : l'agent informe le client par l'état du son contrat.

- **Scénario d'erreur :**

  1. L'acteur pose une question.
  2. Le système fait le traitement nécessaire et demande le code contrat.
  3. Le client envoie son code.
  4. API Restful vérifie le code contrat.
  5. Le système trouve que le code saisi est invalide.
  6. Le système demande à l'acteur de vérifier le code saisi.

5.2.3.4 Diagramme d'activité : Demande un nouveau mot de passe

Après la vérification de l'adresse email, le système envoie une notification au client à fin de l'informer que sa demande de modifier son mot de passe est en cours de traitement.

La figure 5.4 illustre l'activité "Demande de changer mot de passe".

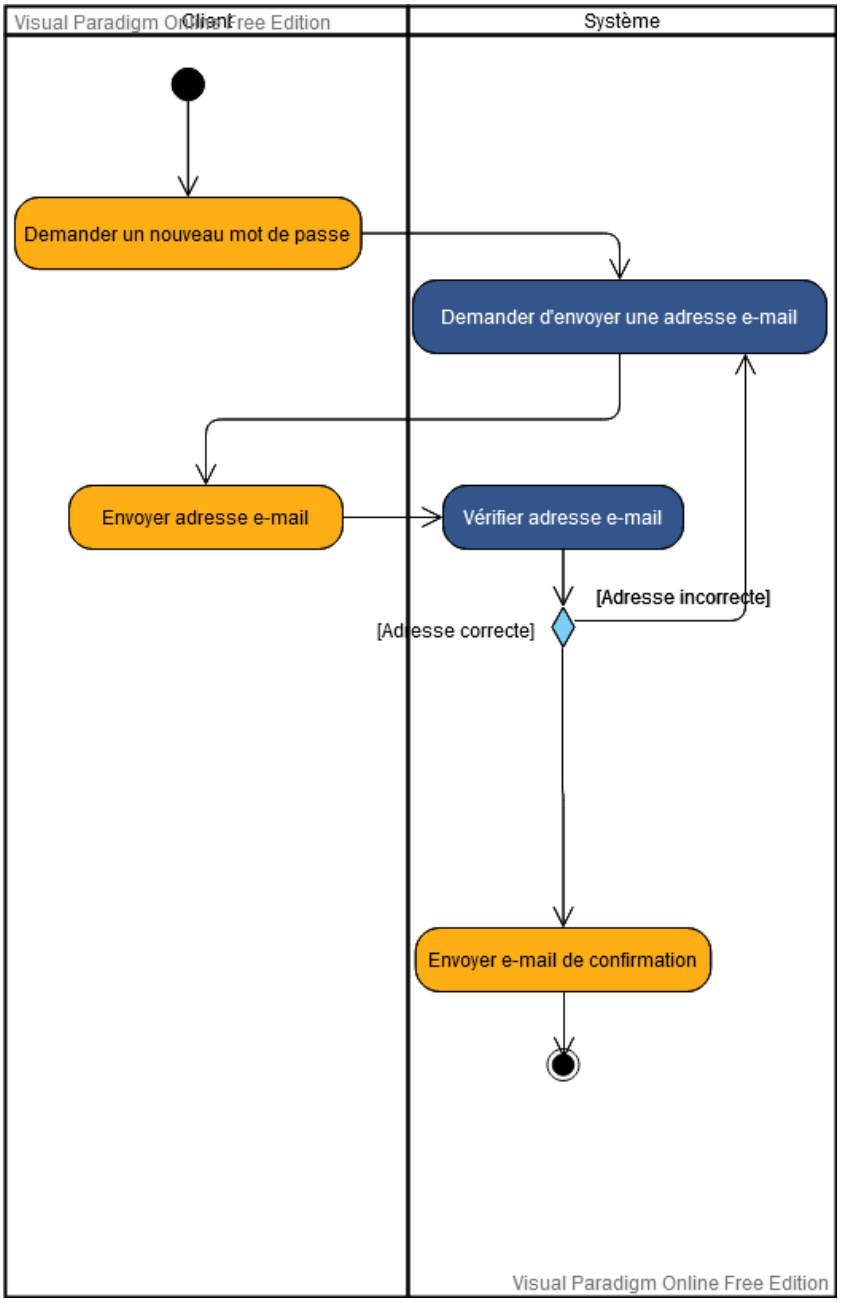


FIGURE 5.4 – Diagramme d'activité : demande un nouveau mot de passe

5.3 Déploiement

Le déploiement de modèles est la méthode utilisée pour intégrer un modèle d'apprentissage automatique dans un environnement de production existant afin de commencer à l'utiliser par l'utilisateur final. Pour cela, cette section est consacrée au développement d'une interface conversationnelle pour que notre projet devienne prêt pour être en production.

5.3.1 Le Système de gestion de base de donnée

Pour notre application le système de gestion de base de données est SQL SERVER. Le système utilise SQL (structured query language) pour définir et manipuler les données . SQL Server prit en compte des spécificités des projets décisionnels et il est très sécurisé possèdent deux modes d'authentification en Windows et SQL SERVER.

5.3.2 Environnement de développement

•Software

Angular :

C'est une plateforme et un cadre permettant de créer des applications client à page unique en utilisant HTML et TypeScript. Elle implémente les fonctionnalités principales et optionnelles sous la forme d'un ensemble de bibliothèques TypeScript. C'est un langage de programmation de scripts utilisé principalement dans le développement des pages web interactives.

Parmi ses avantages, on peut citer :

- Une architecture élégante : c'est à dire une excellente extensibilité nous permet d'ajouter de nouvelles fonctionnalités sans toucher le squelette et bonnes performances sans un long chargement rendant les utilisateurs impatients.

- Une grande communauté, qui permet toujours de pouvoir poser des questions en cas de problème.

- Création d'une application sur une seule page.

- Des améliorations constantes et régulières

Flask :

Un micro-framework de python qui est utilisé dans le développement web et a un syntaxe facile à utiliser. Il est conçu pour rendre la mise en route rapide et facile, avec la possibilité de s'adapter à des applications complexes. Il fournit les outils nécessaires pour créer et déployer une application Web. Ainsi, il n'impose ni des dépendances ni une structure de projet fixe comme les autres frameworks Web Python (exemple :Django).

HTML :

HTML (HyperText Markup Language) langage de balises est utilisé pour faire la création et la représentation du contenu d'une page web et sa structure.

CSS :

(Cascading Style Sheets) consiste à attribuer dans un même document des caractéristiques de mise en forme de page web. Il est utilisé conjointement avec HTML pour modifier le design de la page.

Bootstrap :
Bootstrap est une collection d'outils utiles (Framework) qui permet de créer le design d'application web plus aisément.

SMTP :

SMTP est un protocole utilisé pour transmettre le courrier électronique entre les serveurs.

•Hardware

Visual Studio Code :

Visual Studio Code est un éditeur de code extensible,développé par Microsoft pour Windows. Prenant en charge des centaines de langages on peut citer : Python, Html, CSS, PHP.etc.

Postman :

Un outil utilisé pour les tests fonctionnels des API. Il construit et exécute des requêtes HTTP.Il les stocke dans un historique pour les arranger dans des collections qui permet d'assurer sa fonctionnalité.

5.3.3 Liaison entre la partie Backend et Frontend

Afin que l'agent virtuel reçoive les demandes des clients écrites dans l'interface conversationnelle et pour qu'il effectue les traitements nécessaires pour répondre à ces demandes en temps réel, on doit créer une connexion entre la partie Backend (Flask) et Frontend

(angular).Par conséquent, on a crée une API restful grâce au framework open source pour le développement web Flask. C'est le moyen convenu pour que le front-end et le back-end travaillent ensemble.

•API Restful :

Une API est un ensemble de règles permettant la communication entre différents programmes. Il permet de décrire à travers un standard la manière appropriée d'établir la communication entre les applications pour un développeur de logiciels. Il y a des multiples types d'API et c'est à nous de choisir celle qui s'adapte le mieux à nos besoins. On a opté pour une architecture "Restful".

Une API RESTful est un style architectural pour une interface de programme d'application (API) qui utilise des requêtes HTTP pour accéder aux données et les utiliser. Il existe un ensemble de méthodes de requête qui indiquent l'action que l'on veut réaliser : GET, PUT, POST et DELETE, qui font référence à la lecture, la mise à jour, la création et la suppression des données.

Notre solution suit l'architecture logicielle présentée dans la figure 5.5 qui présente les interactions entre les différents composants de l'application.



FIGURE 5.5 – Architecture logiciel du projet

5.3.4 La liaison entre le chatbot et la base des données de Globalnet

Pour que le chatbot puisse répondre dans un laps de temps court,en d'autre terme pour que la réponse à la requête d'un utilisateur sur un problème soit précise et qu'il réussisse toutes les interactions de vérifications nécessaires avec notre base de données SQL Server, c'est que valide notre choix pour l'architecture de l'api Restful.
La vérification du fonctionnement se fait par une simple requête en utilisant Postman.

La figure 5.8 montre le résultat obtenu pour vérifier l'état d'un contrat pour un client de Globalnet.

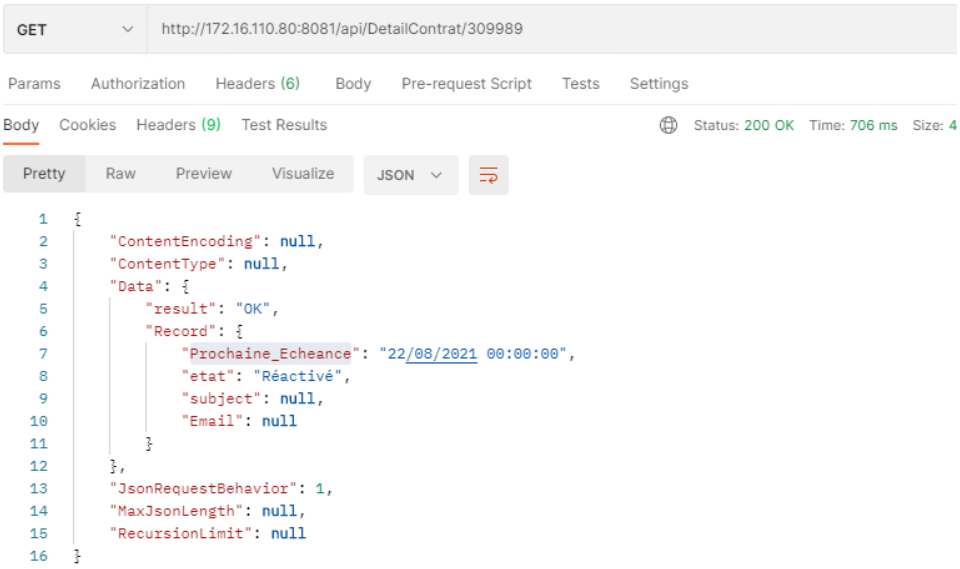


FIGURE 5.6 – Le résultat d'accès au base des données de Globalnet

5.4 Architecture de la solution finale

La figure 5.7 illustre une architecture détaille de Notre Solution intelligente G-nett. On présente les différentes interactions entre les systèmes de base des données, le système de classification contextuel, classification d' intentions et le système d'analyse sentimentale. Tout en précisant, les différentes fonctionnalités de système quelque soit notification par e-mail, enregistrement des conversations dans une base de données.

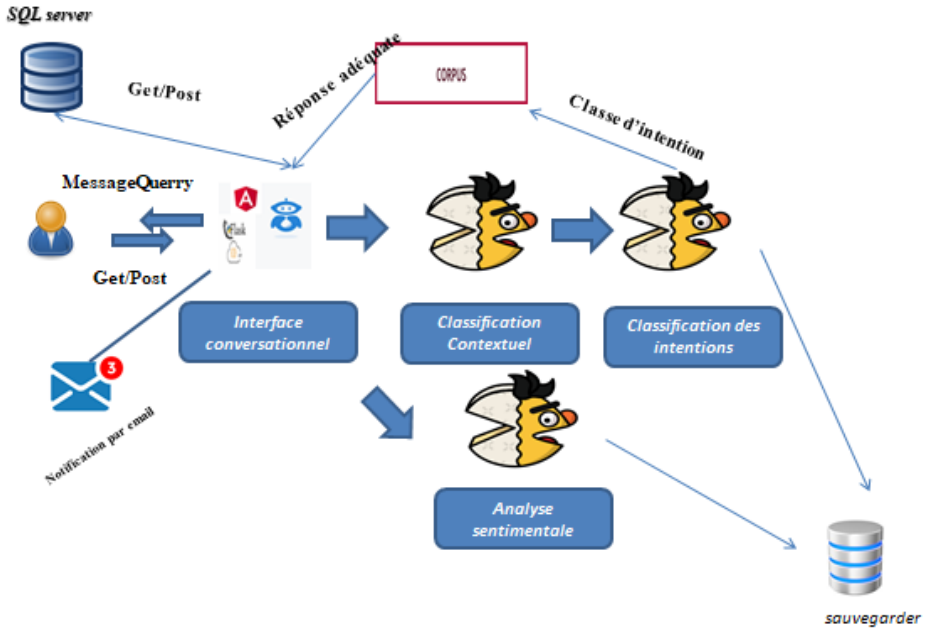


FIGURE 5.7 – Système agent intelligent

La figure 5.8 présente l'interface conversationnelle.

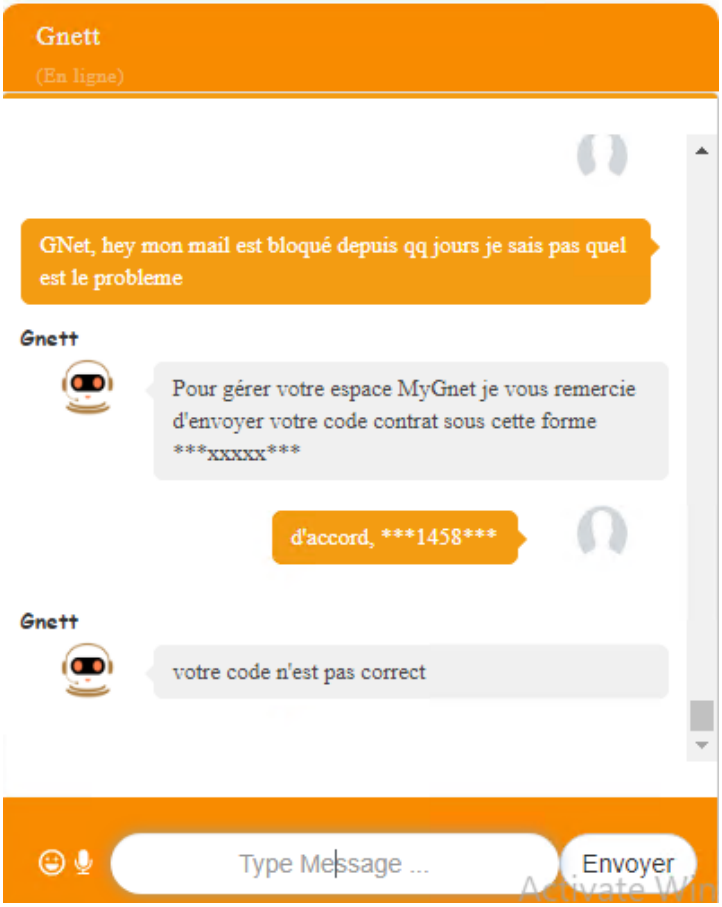


FIGURE 5.8 – L'agent Virtuel G-nett

5.5 Conclusion

Durant ce chapitre, on a commencé par une analyse de spécification de l'interface conversationnelle, suivie par une présentation de quelques diagrammes UML. Ensuite, on a défini l'environnement technique ainsi que les outils utilisés. Finalement, on a visualisé la solution développée en utilisant flask pour le backend et angular pour le frontend.

Conclusion Générale

Ce projet avait pour objectif d'implémenter un agent virtuel avec une précision ,de classification d'intention élevée pour le service client d'un fournisseur d'accès. Cet agent supporte un module d'analyse sentimentale, en exploitant les techniques de traitement du langage naturel et de l'apprentissage automatique.

A travers ce stage , on a cherché à répondre à notre question principale qui est la suivante "comment appliquer les méthodes établies de classification d'intention et d'analyse sentimentale sur des énoncés typiques des demandes clientèles de Globalnet ?"

Pour tenter de répondre à cette question, les performances de trois types de classifieurs différents reconnus pour leur efficacité, ont été évaluées sur un ensemble de données collecté dans le cadre de ce projet : Classifiers basés sur quatres modèles de machine Learning avec TF-idf (SVM, Naive Bayes,régression Logistique et Random Forest) Classifieurs basés sur des modèles neuronaux récurrents (LSTM et Bi-Lstm avec CBOW) Classifieurs basés sur des modèles neuronaux contextuels dynamiques : BERT (version française et multi-langues),

Les aspects sociétaux des résultats de ce projet sont principalement liés au développement continu du traitement de langage naturel, notamment avec l'utilisation accrue des chatbots. Une amélioration générale du comportement des chatbots et du post-traitement des conversations produites pourrait conduire à une mise en œuvre accrue et à une compréhension plus profonde des corpus des conversations existantes.

Au cours de ce projet, on a utilisé les dernières avancées technologiques en NLP, pour la tâche de classification textuelle. L'application des modèles de langage pré-entraînés nouvellement proposés a montré que l'apprentissage par transfert peut être utilisé pour surpasser l'apprentissage traditionnel. Le modèle de langage pré-entraîné CamemBERT obtient les meilleurs résultats en termes de F1-score et de Accuarcy.

Auparavant, les assistants chez Globalnet traitent de grandes quantités de demandes chaque jour, avec des messages provenant de différents canaux. Le traitement manuel

prendrait beaucoup de temps, ferait grimper les coûts, et finirait par impacter négativement la satisfaction des clients. En se basant sur les résultats de ce projet, l'expérience clientèle chez Gloablnet sera enrichie et des réponses de qualité seront assurées à une majorité de clients dans un délai minimum.Par ailleurs, le chatbot promis à l'issue de ce stage garantit une fidélisation client.

Notre projet est susceptible d'être amélioré et évolué. En effet, notre travail n'est que le début d'un long processus.
Dans un travail future de recherche, on essaye de rendre le chatbot de Globalnet hybride. Ce chatbot aura donc la possibilité de donner des réponses personnalisées et automatiques à travers l'utilisation de deux approches en même temps : l'approche classique de classification et l'approche de génération des réponses (utilisation éventuelle de la technique de GPT).
En outre, on a envisagé d'ajouter un module de recommandation des produits et des offres au sein de Chatbot. Puis, afin de rendre l'agent virtuel universel, on a prévu d'implémenter un module de traduction neuronale moderne multilingue basé sur le "Transformers".

Annexes

Le modèle CamemBERT

Poids de CamemBERT : la figure A.1 montre les paramètres de la classe Camembert-ForsequenceClassification.

```
CamembertForSequenceClassification(
  (roberta): RobertaModel(
    (embeddings): RobertaEmbeddings(
      (word_embeddings): Embedding(32005, 768, padding_idx=1)
      (position_embeddings): Embedding(514, 768, padding_idx=1)
      (token_type_embeddings): Embedding(1, 768)
      (LayerNorm): LayerNorm((768,), eps=1e-05, elementwise_affine=True)
      (dropout): Dropout(p=0.1, inplace=False)
    )
    (encoder): RobertaEncoder(
      (layer): ModuleList(
        (0): RobertaLayer(
          (attention): RobertaAttention(
            (self): RobertaSelfAttention(
              (query): Linear(in_features=768, out_features=768, bias=True)
              (key): Linear(in_features=768, out_features=768, bias=True)
              (value): Linear(in_features=768, out_features=768, bias=True)
              (dropout): Dropout(p=0.1, inplace=False)
            )
            (output): RobertaSelfOutput(
              (dense): Linear(in_features=768, out_features=768, bias=True)
              (LayerNorm): LayerNorm((768,), eps=1e-05, elementwise_affine=True)
```

FIGURE A.1 – CamemBERTForSequenceClassification

Le modèle BERT

Poids de BERT : la figure suivante présente les paramètres de la classe BERTForSequenceClassification.

```
BertForSequenceClassification(
  (bert): BertModel(
    (embeddings): BertEmbeddings(
      (word_embeddings): Embedding(105879, 768, padding_idx=0)
      (position_embeddings): Embedding(512, 768)
      (token_type_embeddings): Embedding(2, 768)
      (LayerNorm): LayerNorm((768,), eps=1e-12, elementwise_affine=True)
      (dropout): Dropout(p=0.1, inplace=False)
    )
    (encoder): BertEncoder(
      (layer): ModuleList(
        (0): BertLayer(
          (attention): BertAttention(
            (self): BertSelfAttention(
              (query): Linear(in_features=768, out_features=768, bias=True)
              (key): Linear(in_features=768, out_features=768, bias=True)
              (value): Linear(in_features=768, out_features=768, bias=True)
              (dropout): Dropout(p=0.1, inplace=False)
            )
            (output): BertSelfOutput(
              (dense): Linear(in_features=768, out_features=768, bias=True)
              (LayerNorm): LayerNorm((768,), eps=1e-12, elementwise_affine=True)
              (dropout): Dropout(p=0.1, inplace=False)
```

FIGURE A.2 – BERTForSquenceClassification

Mécanisme d'attention

Self attention On décrit le mécanisme d'attention de la section 2.8.1 : La première étape du calcul de le self attention consiste à extraire à partir de vecteur d'entrée de l'encoder trois vecteurs.

Dans une séquence de texte notée X = $\begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_i \\ \vdots \\ x_N \end{pmatrix}$

est une matrice de taille (N × M) avec N est la longueur de la séquence et M est la dimension d'embedding (chaque mot est représenté par un vecteur de taille M que l'on note Xi). Pour chaque mot Xi, on crée 3 vecteurs, un vecteur "Query", un vecteur "Key", et un vecteur "Value" que l'on note respectivement Qi, Ki, Vi. Ils sont des abstractions qui sont utiles pour le calcul et la compréhension de l'attention. Ces vecteurs sont obtenus en multipliant l'embedding de la séquence d'entrée (X_i) par les trois matrices de poids (W^Q, W^K, W^V) qui sont entraînées pendant le processus d'entraînement du Transformer.

$$Q_i = X_i * W^Q$$

$$K_i = X_i * W^K$$

$$V_i = X_i * V^K$$

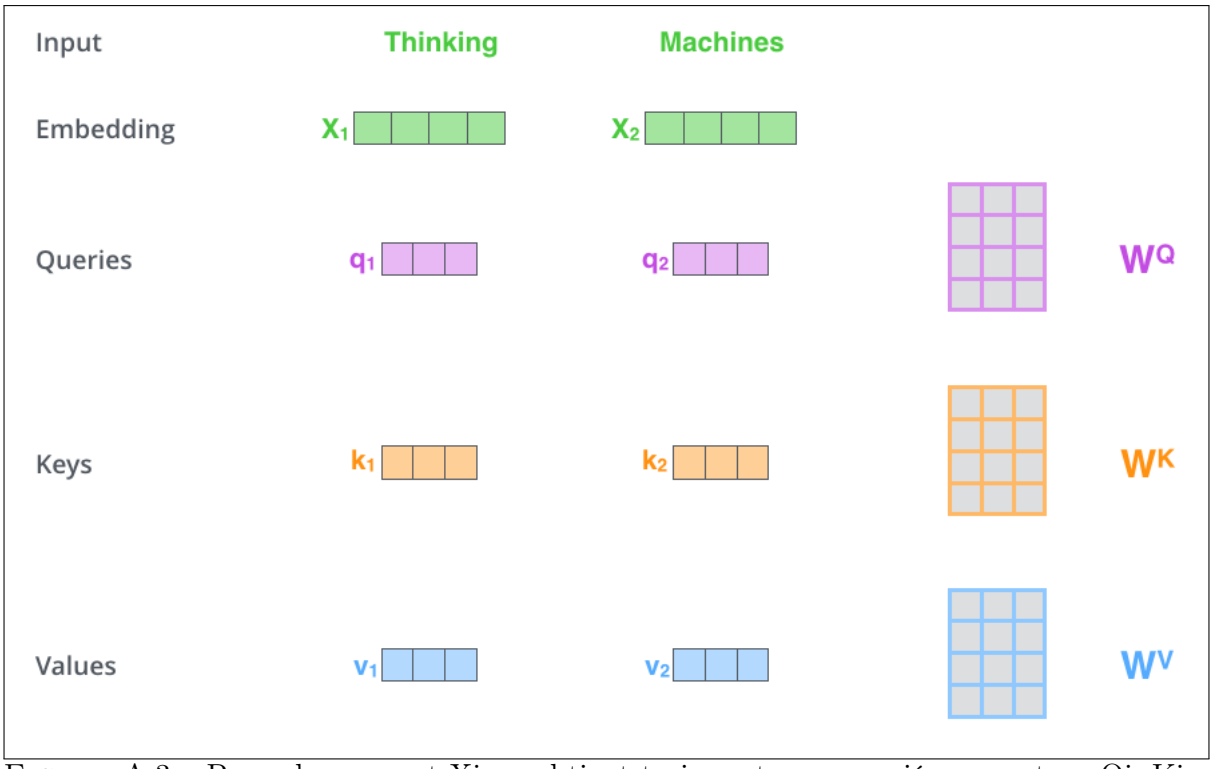


FIGURE A.3 – Pour chaque mot Xi on obtient trois vecteurs associés au vecteur Qi, Ki, Vi

La valeur de self attention est définie par la formule suivante :

$$Attention(Q, K, V) = softmax(\frac{QK^T}{\sqrt{d_k}})V$$

dk : c'est la racine de la dimension de K qui dans l'article scientique [32] est fixé à 64. L'idée derrière l'application de la fonction Softmax est de garder intactes les valeurs des mots sur lesquels on veut se concentrer, et de donner un poids négligeable aux mots non pertinents.
La figure ci-dessous présente le mécanisme de self attention qui est appliqué à une seule séquence.

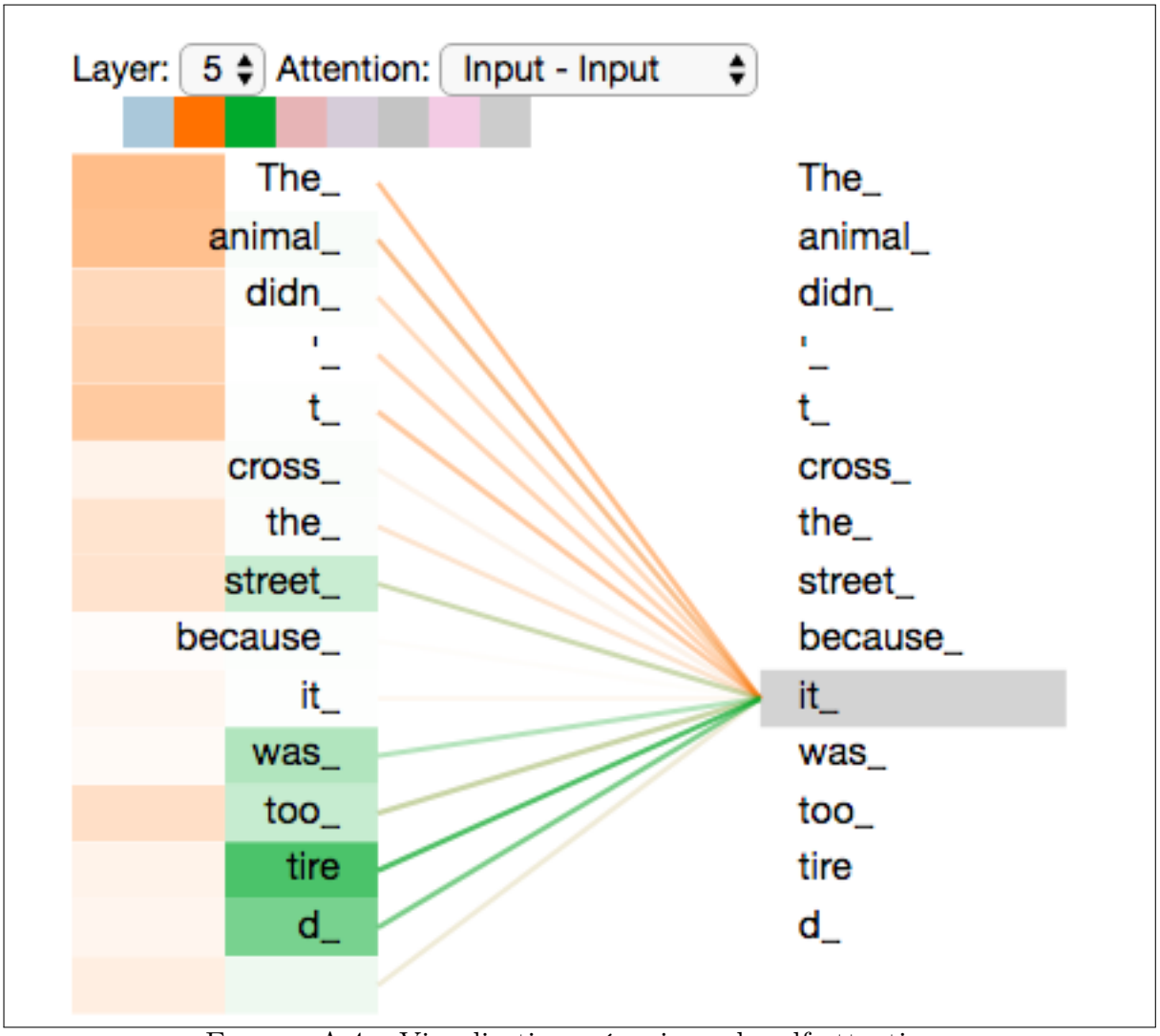


FIGURE A.4 – Visualisation mécanisme de self attention

La couche de self-attention détermine l'interdépendance des différents mots de la même séquence pour lui associer une représentation pertinente. Prenons par exemple la phrase : "The animal didn't cross the street because it was too tired" .C'est clair, pour un être humain, que "it" fait référence au " The animal", mais c'est pas le cas pour une machine. Donc le processus de self-attention aura pour objectif de détecter le lien entre "animal" et "it".
MultiHead attention Dans Bert/CamemBERT, le processus d'attention se fait en paral-

lèle grâce à "MultiHead attention", qui calcule le self attention en parallèle pour toute la séquence. MultiHead attention est représenté par l'équation suivante :

$$MultiHead(Q, K, V) = Concat(head_1, ..., head_h)W^O$$

avec :

$$head_i = Attention(QW_i^Q, KW_i^K, VW_i^V)$$

Bibliographie

[1] Understanding lstm networks. 2015.

[2] Site Web Hugging Face. 2016.

[3] Bidirectional lstm. 2018.

[4] Linkedin de 3S. 2018.

[5] Simage classification : Mlp vs cnn). 2019.

[6] Site Web Towards Data Science. 2019.

[7] Site web figshare. 2020.

[8] Support vector machines (svm). 2020.

[9] Rnn – architecture, mapping, and propagation. 2021.

[10] 3S. Site web 3s. 2017.

[11] Cho Kyunghyun Bahdanau, Dzmitry and Yoshua Bengio. Neural machine translation by jointly learning to align and translate. 2015.

[12] Ducharme Réjean Bengio, Yoshua and Pascal. Vincent. A Neural Probabilistic Language Model. 2000.

[13] Kenton Lee cob Devlin, Ming-Wei Chang, Jones Llion Gomez Aidan N. Kaiser Lukasz Kristina Toutanova., Jakob, and Illia. Polosukhin. BERT : Pre-training of deep bidirectional transformers for language understanding. 2018.

[14] dward Ma. Nlpaug. 2019.

[15] Zhang et al. Character-level Convolutional Networks for TextClassification. 2016.

[16] Alex Graves, Uszkoreit Jakob Jones Llion Gomez Aidan N. Kaiser Lukasz Schmidhuber, Jürgen., and Illia. Polosukhin. Framewise Phoneme Classification withBidirectional LSTM Networks. 2005.

[17] Frank Hutter Ilya Loshchilov. Decoupled Weight Decay Regularization. 2017.

[18] Diederik P Kingma and Jimmy. Ba. Adam : A method for stochastic optimization. 2014.

[19] Sosuke Kobayashi. Contextual Augmentation : Data Augmentation by Words with Paradigmatic Relations. 2018.

[20] Richardson John. Kudo Taku. Sentencepiece : A simple and language independent subword tokenizer and detokenizer for neural text processing. 2018.

[21] Yinhan et al. Liu. RoBERTa : A Robustly Optimized BERT Pretraining Approach. 2019.

[22] Pedro Javier Ortiz Suárez Yoann Dupont Laurent Romary Éric Villemonte de la Clergerie Djamé Seddah Benoît Sagot. Louis Martin, Benjamin Muller. CamemBERT : a Tasty French Language Model. 2019.

[23] Chris Manna. An intro to the crisp-dm methodology. 2019.

[24] Louis Martin. CamemBERT : a Tasty French Language Model. 2019.

[25] Christof Monz Marzieh Fadaee, Arianna Bisazza. Data Augmentation for Low-Resource Neural Machine Translation. 2017.

[26] Sutskever Ilya Chen Kai Corrado Greg Mikolov, Tomas and Jeffrey.R Dean. Distributed Representations of Words and Phrases and their Compositionality. 2013.

[27] Rodrigo Pimentel. Chatito. 2018.

[28] Sebastian Raschka. Why is logistic regression considered a linear model. 2013.

[29] Haddow Barry Sennrich, Rico and Alexandra. Birch. Neural Machine Translation of Rare Words with Subword Units. 2016.

[30] Jürgen Schmidhuber Sepp Hochreiter. Long short-term memory. 1997.

[31] Spacy. Why is logistic regression considered a linear model. 2013.

[32] Shazeer Noam Parmar Niki Uszkoreit Jakob Jones Llion Gomez Aidan N. Kaiser Lukasz Vaswani, Ashish and Illia. Polosukhin. Attention Is All You Need. 2017.

[33] Diyi Yang William Yang Wang. Proceedings of the 2015 Conference on Empirical Methods in Natural Language Processing. 2015.

[34] Mike Schuster et al. Yonghui Wu. Google's Neural Machine Translation System : Bridging the Gap between Human and Machine Translation. 2016.